

Algoritmi paraleli și distribuiți

Suport de curs

Mitică Craus, Alexandru Archip

Universitatea Tehnică „Gheorghe Asachi” din Iași

Pe scurt...

1. Introducere
2. Modele de comunicare
3. Topologii de comunicare
4. Modele de calcul paralel/distribuit
5. Parametri de performanță a unui algoritm paralel/distribuit
6. Comunicarea în sistemele de calcul paralel/distribuit
7. Algoritmi fundamentali
8. Sortare paralelă
9. Calcul matriceal
10. Sisteme de ecuații liniare
11. Alegerea liderului
12. Excluderea mutuală în sisteme *shared-memory*

Cuprins

1. Introducere
2. Modele de comunicare
3. Topologii de comunicare
4. Modele de calcul paralel/distribuit
5. Parametri de performanță a unui algoritm paralel/distribuit
6. Comunicarea în sistemele de calcul paralel/distribuit
7. Algoritmi fundamentali
8. Sortare paralelă
9. Calcul matriceal
10. Sisteme de ecuații liniare
11. Alegerea liderului
12. Excluderea mutuală în sisteme *shared-memory*

Sisteme abstracte de calcul paralel/distribuit

- ▶ Un sistem abstract de calcul paralel/distribuit (SACP/D) este un ansamblu de unități de procesare a datelor, care comunică între ele.
- ▶ Unitatea de procesare este un element atomic al sistemului, cu o structură internă invizibilă.
- ▶ Se remarcă în această definiție două componente de bază ale unui SACP/D:
 - ▶ componenta computațională;
 - ▶ componenta comunicațională.

Arhitecturi paralele/distribuite

- ▶ O arhitectură paralelă/distribuită este o implementare a unui SACP/D. Aceasta înseamnă:
 - ▶ maparea subsistemului computațional pe o mulțime de mașini cu o funcționalitate bine definită și
 - ▶ asignarea relațiilor de comunicare la un model de comunicare.
- ▶ Modelul de comunicare se referă la mediul de comunicare și modalitatea în care unitățile de procesare comunică între ele.
- ▶ O arhitectură paralelă/distribuită execută algoritmi paraleli/distribuiți.
- ▶ Definiția unui algoritm paralel/distribuit poate fi derivată din definiția unui algoritm secvențial, prin înlocuirea noțiunii de pas secvențial cu cea de pas paralel.
- ▶ Pasul paralel poate fi un pas de sincronizare (pas paralel impropriu) sau un ansamblu de pași secvențiali, executați în paralel în unitățile de procesare (pas paralel propriu).

Observație

Paralelismul implică distribuție, dar distribuția nu implică paralelism. Acesta ar putea fi punctul de plecare în diferențierea algoritmilor paraleli de cei distribuiți.

Cuprins

1. Introducere
2. **Modele de comunicare**
3. Topologii de comunicare
4. Modele de calcul paralel/distribuit
5. Parametri de performanță a unui algoritm paralel/distribuit
6. Comunicarea în sistemele de calcul paralel/distribuit
7. Algoritmi fundamentali
8. Sortare paralelă
9. Calcul matriceal
10. Sisteme de ecuații liniare
11. Alegerea liderului
12. Excluderea mutuală în sisteme *shared-memory*

Taxonomie

- ▶ Determinant în definirea unui model de comunicare este mediul de comunicare.
- ▶ Modelele de comunicare universal acceptate sunt cele bazate pe memoria partajată și rețelele de interconectare.

Modelul de comunicare bazat pe memorie partajată

- ▶ Are la bază principiul comunicării prin variabile de memorie.
- ▶ Topologia de comunicare are potențialul unui graf complet, deși acesta nu este decât o stea specială în care memoria comună este nodul central.
- ▶ Oricare două unități de procesare pot comunica doar prin intermediul memoriei comune (centrul steii). Aceasta reprezintă o vulnerabilitate.

Modelul de comunicare bazat pe rețele de interconectare

- ▶ Oricare două unități de procesare comunică între ele prin canalul de comunicații care le leagă sau prin intermediul unei secvențe de canale care are la capete cele două unități de procesare.
- ▶ Un astfel de model poate fi definit ca fiind format din:
 - ▶ un graf $G = (V, E)$, care reprezintă topologia de comunicare și
 - ▶ o funcție $c : E \rightarrow C$, unde C este o mulțime de capacități de comunicare.

Modelul de comunicare bazat pe memorie partajată

- (a) memorie locală privată și memorie externă partajată;
- (b) memorie locală privată+partajată și memorie externă partajată;
- (c) memorie locală privată+partajată.

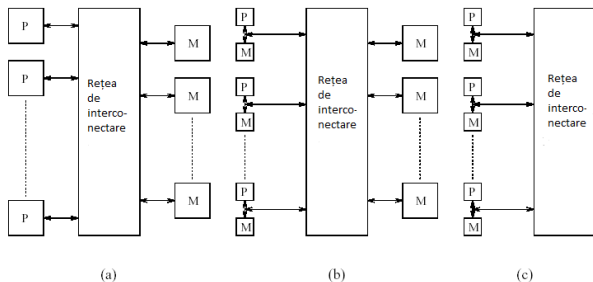


Figura 1: Modelul de comunicare bazat pe memorie partajată

Modelul de comunicare bazat pe rețele de interconectare

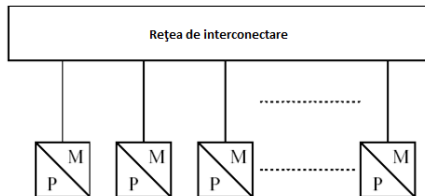


Figura 2: Modelul de comunicare bazat pe rețele de interconectare

Cuprins

1. Introducere
2. Modele de comunicare
- 3. Topologii de comunicare**
4. Modele de calcul paralel/distribuit
5. Parametri de performanță a unui algoritm paralel/distribuit
6. Comunicarea în sistemele de calcul paralel/distribuit
7. Algoritmi fundamentali
8. Sortare paralelă
9. Calcul matriceal
10. Sisteme de ecuații liniare
11. Alegerea liderului
12. Excluderea mutuală în sisteme *shared-memory*

Topologii de comunicare

Clasificare

Topologii regulate:

- ▶ graful complet;
- ▶ lanțul;
- ▶ inelul;
- ▶ steaua;
- ▶ arborele binar complet;
- ▶ arborele gras;
- ▶ fluturele;
- ▶ banyan
- ▶ plasa;
- ▶ plasa de arbori;
- ▶ hipercubul;
- ▶ amestecul perfect.

Topologii neregulate:

- ▶ graful oarecare;
- ▶ arborele.

Graful complet

- Rețelele peer-to-peer.

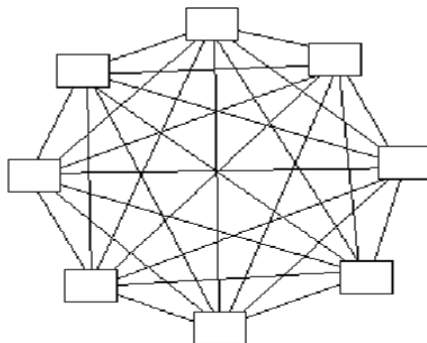


Figura 3: Graful complet

Lanțul

- ▶ Mulțimea nodurilor V este organizată sub forma unui tablou unidimensional de dimensiune n , prin intermediul unei funcții de indexare $I: V \rightarrow L = \{0, 1, \dots, n-1\}$.
- ▶ Dacă se notează cu $V_i = I^{-1}(i)$, nodul V_i este conectat cu nodurile V_{i-1} și V_{i+1} . Fac excepție nodurile $V_0 = I^{-1}(0)$ și $V_{n-1} = I^{-1}(n-1)$. Nodul V_0 este conectat doar cu nodul V_1 . Nodul V_{n-1} este conectat doar cu nodul V_{n-2} .



Figura 4: Lanțul

Inelul

- ▶ Mulțimea nodurilor V este indexată prin intermediul unei funcții $I: V \rightarrow L = \{0, 1, \dots, n-1\}$.
- ▶ Nodul $V_i = I^{-1}(i)$ are vecini nodurile $V_{i \ominus 1}$ și $V_{i \oplus 1}$, unde $\ominus$ este scăderea modulo n , iar $\oplus$ este adunarea modulo n .

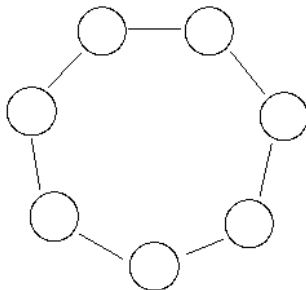


Figura 5: Inelul

Steaua

- ▶ Mulțimea nodurilor, V , este indexată prin intermediul unei funcții $I: V \rightarrow L = \{0, 1, \dots, n-1\}$.
- ▶ Există un nodul special, V_0 , la care sunt conectate toate nodurile.
- ▶ Comunicarea între noduri se realizează prin intermediul nodului V_0 .
- ▶ Steaua este topologia specifică modelului de comunicare prin intermediul memoriei comune.

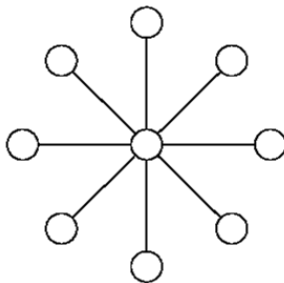


Figura 6: Steaua

Arborele binar complet

- ▶ Mulțimea nodurilor V este indexată prin intermediul unei funcții $I: V \rightarrow L = \{0, 1, \dots, n-1\}$.
- ▶ Există un nodul special, V_0 , numit rădăcină. Rădăcina nu are părinte și are doi fii, V_1 și V_2 .
- ▶ Fiecare nod interior are un părinte și doi fii. Părintele unui nod interior $V_i = I^{-1}(i)$ este $V_{[(i-1)/2]}$. Fiii sunt V_{2i+1} și V_{2i+2} .
- ▶ Frunzele au un părinte, nu au fii și sunt situate pe același nivel. Părintele unei frunze $V_i = I^{-1}(i)$ este $V_{[(i-1)/2]}$.
- ▶ Între oricare două noduri există un singur drum. Numărul conexiunilor este $n-1$.

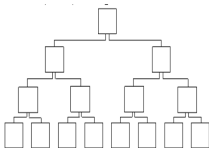


Figura 7: Arborele binar complet

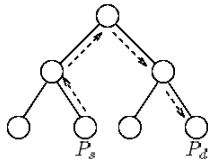


Figura 8: Comunicarea

Arborele gras

- ▶ Arbore binar complet.
- ▶ Fiecare pereche de noduri dispune de un canal de comunicare propriu.
- ▶ Numărul de muchii care leagă un nod interior de fii este egal cu $2^{h-nivel-1}$, unde h este adâncimea arborelui, iar $nivel(radacina) = 1$, $nivel(frunza) = h$.

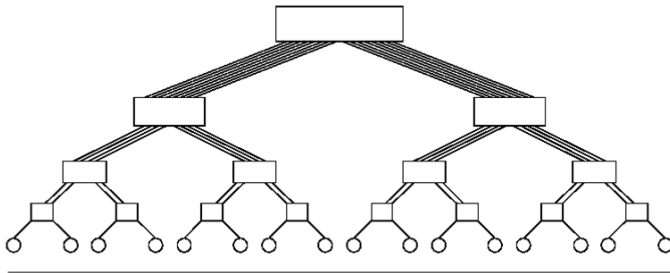


Figura 9: Arborele gras

Fluturile

- ▶ Arbore cu mai multe rădăcini!
- ▶ Exact un drum între oricare două noduri de pe frontieră.
- ▶ Mulțimea nodurilor V este organizată ca un tablou bidimensional de dimensiune $n \times m$, $n = 2^{m-1}$, prin intermediul unei funcții de indexare $I: V \rightarrow L = \{(i, j) / 0 \leq i < n, 0 \leq j < m\}$.
- ▶ Notăm cu $V_{i,j}$ nodul dat de $I^{-1}(i, j)$. Nodul $V_{i,j}$, situat pe coloana j este conectat la coloana $j+1$ prin nodurile $V_{i,j+1}$ și $V_{i \oplus 2^{m-j-2} + [i/2^{m-j-1}] \times 2^{m-j-1}, j+1}$, unde $\oplus$ este adunarea modulo 2^{m-j-1} .

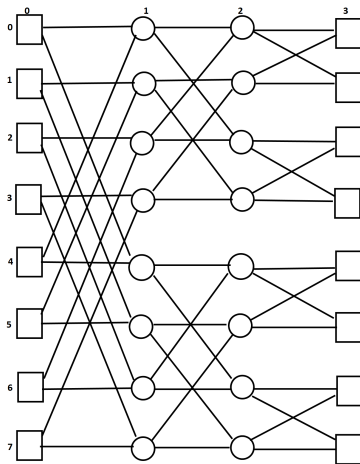


Figura 10: Fluturile

Topologia Banyan

- ▶ Tot arbore cu mai multe rădăcini!
- ▶ Mulțimea nodurilor V este organizată ca un tablou bidimensional de dimensiune $n \times m$, $n = 2^{m-1}$, prin intermediul unei funcții de indexare $I: V \rightarrow L = \{(i, j) / 0 \leq i < n, 0 \leq j < m\}$.
- ▶ Notăm cu $V_{i,j}$ nodul dat de $I^{-1}(i, j)$. Nodul $V_{i,j}$, situat pe coloana j este conectat la coloana $j+1$ prin nodurile $V_{[i/2] \oplus 0 + [i/2^{m-j-1}] \times 2^{m-j-1}, j+1}$ și $V_{[i/2] \oplus 2^{m-j-2} + [i/2^{m-j-1}] \times 2^{m-j-1}, j+1}$, unde $\oplus$ este adunarea modulo 2^{m-j-1} .

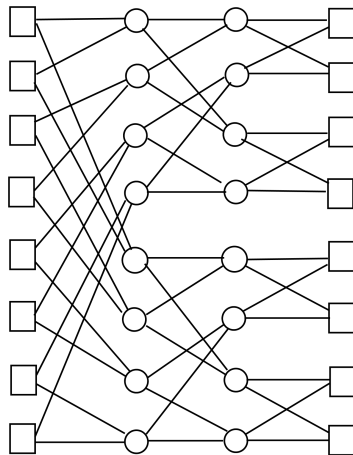


Figura 11: Topologia Banyan

Plasa

- ▶ Este o generalizare a topologiei de tip lanț.
- ▶ Mulțimea nodurilor V este organizată ca un tablou k -dimensional prin intermediul unei funcții de indexare
 $I: V \rightarrow L = \{(i_{k-1}, i_{k-2}, \dots, i_0) / 0 \leq i_j < n_j, j = k-1, k-2, \dots, 0\}$, unde n_j este mărimea celei de-a j -a dimensiuni, iar $n = n_{k-1} * n_{k-2} \cdots * n_0$.
- ▶ Notăm cu $V_{i_{k-1}, i_{k-2}, \dots, i_0}$ nodul dat de $I^{-1}(i_{k-1}, i_{k-2}, \dots, i_0)$.
- ▶ Nodul $V_{i_{k-1}, i_{k-2}, \dots, i_j, \dots, i_0}$ este adiacent cu nodul $V_{i_{k-1}, i_{k-2}, \dots, i_j+1, \dots, i_0}$, dacă $i_j + 1 < n_j$.
- ▶ Nodul $V_{i_{k-1}, i_{k-2}, \dots, i_j, \dots, i_0}$ este adiacent cu nodul $V_{i_{k-1}, i_{k-2}, \dots, i_j-1, \dots, i_0}$, dacă $i_j - 1 \geq 0$.

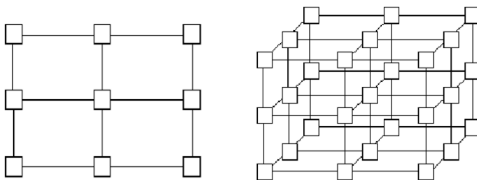


Figura 12: Plasa

Plasa circulară

- ▶ Este o generalizare a topologiei de tip inel.
- ▶ Nodul $V_{i_{k-1}, i_{k-2}, \dots, i_j, \dots, i_0}$ este adiacent cu nodul $V_{i_{k-1}, i_{k-2}, \dots, i_j \oplus 1, \dots, i_0}$, unde $\oplus$ este adunarea modulo n_j .
- ▶ Nodul $V_{i_{k-1}, i_{k-2}, \dots, i_j, \dots, i_0}$ este adiacent cu nodul $V_{i_{k-1}, i_{k-2}, \dots, i_j \ominus 1, \dots, i_0}$, unde $\ominus$ este scăderea modulo n_j .

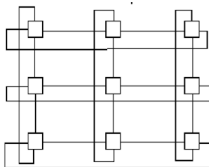


Figura 13: Plasa circulară

Plasa de arbori

- ▶ Mulțimea nodurilor, V este partiționată în două submulțimi M și T
- ▶ Nodurile din M sunt organizate sub forma unei matrice.
- ▶ Conectarea nodurilor din M se realizează prin intermediul unor arbori binari formați din noduri din mulțimea T , câte un arbore pentru fiecare linie și fiecare coloană.
- ▶ Nodurile din M aparținând unei linii/coloane constituie frunzele arborelui asociat liniei/coloanei respective.

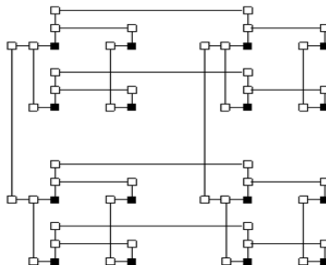


Figura 14: Plasa de arbori

Hipercubul (Cubul cu k dimensiuni)

- Numărul n al nodurilor grafului ce reprezintă o astfel de topologie este 2^k , unde k este dimensiunea cubului.
- Fie $i_{k-1}i_{k-2}\dots i_0$ reprezentarea binară a lui $i \in \{0, \dots, n-1\}$ și fie $i^{(j)}$ numărul a cărui reprezentare binară este $i_{k-1}i_{k-2}\dots i_{j+1}\tilde{i}_ji_{j-1}\dots 0$, unde $\tilde{i}_j = 1 - i_j, 0 \leq j < k$.
- Nodul V_i este adiacent cu nodurile din mulțimea $\{V_{i^{(j)}}; 0 \leq j < k\}$.

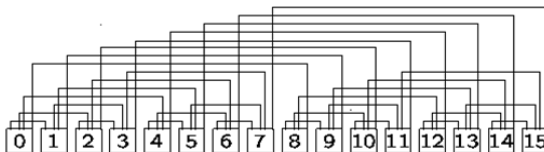


Figura 15: Hipercub liniarizat

Hipercuburi cu 0, 1, 2 , 3 și 4 dimensiuni

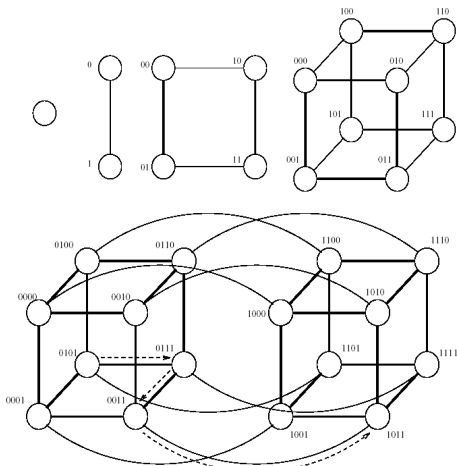


Figura 16: Hipercuburi cu 0, 1, 2 , 3 și 4 dimensiuni

Amestecul perfect

- Fie $i_{k-1}i_{k-2}\dots i_0$ reprezentarea binară a lui $i \in \{0, \dots, n-1\}$.
- Nodul V_i este conectat la nodurile $V_{i^{(0)}}$, $V_{i_{k-2}i_{k-3}\dots i_0 i_{k-1}}$ și $V_{i_0 i_{k-1} i_{k-2} \dots i_1}$, $0 \leq i < n$,
- n, k, i și $i^{(0)}$ sunt definite ca în cazul hipercubului.

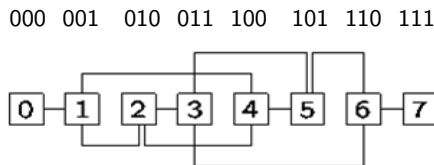


Figura 17: Amestec perfect

Criterii de evaluare a topologiilor regulate

- ▶ Distanța = cel mai scurt drum între două unități de procesare.
- ▶ Diametrul = distanța maximă între două unități de procesare.
- ▶ Lățimea biseției = numărul minim de linii de comunicații care pot fi întrerupte pentru a sparge rețeaua în două rețele de dimensiuni aproximativ egale.
- ▶ Costul = numărul de linii de comunicații.

Evaluarea a 8 topologii regulate, cu p unități de procesare

Topologia	Diametrul	Lățimea biseției	Costul
Graful complet	1	$\lceil p^2/4 \rceil$	$p(p-1)/2$
Steaua	2		$p-1$
Arborele binar complet	$2\lceil \log(p+1)/2 \rceil$	1	$p-1$
Lanțul	$p-1$	1	$p-1$
Inelul	$\lceil p/2 \rceil$	2	p
Plasa 2D	$2(\sqrt{p}-1)$	$\sqrt{p}$	$2(p-\sqrt{p})$
Plasa 2D ciclică	$2\lceil (\sqrt{p}/2) \rceil$	$2\sqrt{p}$	$2p$
Hipercubul	$\log p$	$p/2$	$p \log p/2$

Graful oarecare

► Internet-ul.

► Web-ul.

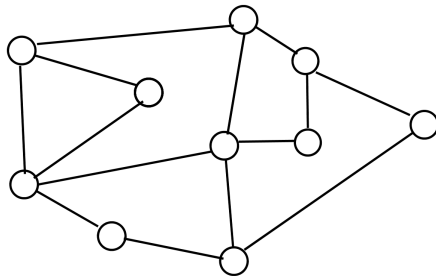


Figura 18: Graful oarecare

Arborele oarecare

- rețelele locale.

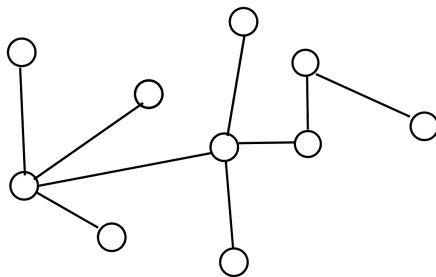


Figura 19: Arborele oarecare

Cuprins

1. Introducere
2. Modele de comunicare
3. Topologii de comunicare
- 4. Modele de calcul paralel/distribuit**
5. Parametri de performanță a unui algoritm paralel/distribuit
6. Comunicarea în sistemele de calcul paralel/distribuit
7. Algoritmi fundamentali
8. Sortare paralelă
9. Calcul matriceal
10. Sisteme de ecuații liniare
11. Alegerea liderului
12. Excluderea mutuală în sisteme *shared-memory*

Modele de calcul paralel/distribuit

Taxonomia lui Flynn

- ▶ Formulată de Michael J. Flynn, în 1966. Criteriul de clasificare este concurența instrucțiunilor și a fluxurilor de date.
- ▶ Un calculator monoprosesor poate executa la un moment dat o singură instrucțiune, în timp ce un sistem multiprosesor poate executa simultan mai multe instrucțiuni asupra mai multor fluxuri de date.
- ▶ Taxonomia:
 - ▶ **SISD** Single Instruction / Single Data stream (o singură instrucțiune / un singur flux de date). Exemplu: calculatoarele monoprosesor;
 - ▶ **SIMD** Single Instruction / Multiple Data streams (o singură instrucțiune / mai multe fluxuri de date). Exemplu: calculatoarele vectoriale, sistemele GPU;
 - ▶ **MISD** Multiple Instructions / Single Data stream (mai multe instrucțiuni / un singur flux de date). Exemplu: sistemele *pipeline*;
 - ▶ **MIMD** Multiple Instructions / Multiple Data streams (mai multe instrucțiuni / mai multe fluxuri de date). Exemplu: calculatoarele multiprosesor, clusterelor, *grid*-urile, norii de calcul (*cloud*).
- ▶ Tipuri de sisteme **MIMD**
 - ▶ **SPMD** Single Program / Multiple Data streams (un singur program / mai multe fluxuri de date);
 - ▶ **MPMD** Multiple Program / Multiple Data streams (mai multe programe / mai multe fluxuri de date).

Modele arhitecturale

- ▶ Modele bazate pe comunicarea prin memoria comună (Shared-Memory)
- ▶ Modele bazate pe comunicarea prin mesaje (Message-Passing)

Modelul Shared-Memory

- ▶ Un sistem Shared-Memory este compus din n unități de procesare $p_0, p_1, \dots, p_{n-1}$ și m regiștri de memorie, $r_0, r_1, \dots, r_{m-1}$.
- ▶ Fiecare registru este caracterizat prin:
 - ▶ valoarea care poate fi memorată în registru;
 - ▶ operațiile care pot fi efectuate asupra conținutului registrului;
 - ▶ valoarea returnată de fiecare operație;
 - ▶ valoarea aflată în registru după fiecare operație.
- ▶ Exemplu - registrul *integer-valued read/write*:
 - ▶ Valorile v memorate de registrul r sunt numere întregi.
 - ▶ Operațiile suportate sunt $\text{read}(r, v)$ and $\text{write}(r, v)$.
 - ▶ Operația $\text{read}(r, v)$ întoarce valoarea v și lasă conținutul registrului neschimbat.
 - ▶ Operația $\text{write}(r, v)$ nu returnează nimic, dar schimbă conținutul registrului r , înscrind acolo valoarea v .
- ▶ Evenimente:
 - ▶ Evenimentele sunt *pași computaționali*, executați de unitățile de procesare.
 - ▶ Un pas computațional, executat de o unitate de procesare p_i , este executat astfel:
 - ▶ În funcție de starea sa, p_i selectează un registru r_j și una din operațiile asociate acestui registru.
 - ▶ Execută operația asupra conținutului lui r_j .
 - ▶ Își schimbă starea, în funcție de starea curentă și valoarea returnată de operația executată.

Modelul PRAM

- ▶ Este derivat din modelul Shared-Memory.
- ▶ Unitățile de procesare sunt procesoare simple.
- ▶ Cele $p(n)$ procesoare $P_1, \dots, P_{p(n)}$ comunică între ele prin intermediul regiștrilor de memorie $R_1, \dots, R_{m(n)}$.
- ▶ Procesoarele execută în mod sincron aceeași instrucțiune, sub control central.
- ▶ Deși instrucțiunea este aceeași, regiștrii de memorie asupra cărora acționează procesoarele pot diferi de la un procesor la altul.
- ▶ Conform taxonomiei Flynn, mașina PRAM este un sistem SIMD.

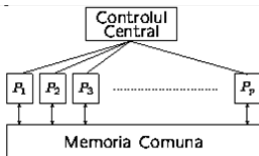


Figura 20: Mașina PRAM

Tipuri de mașini PRAM

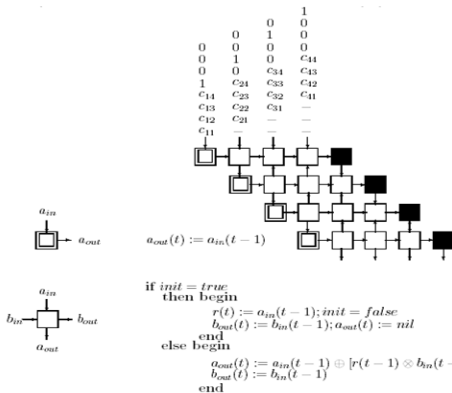
- ▶ La un moment dat t , un procesor poate accesa un registru de memorie pentru citire sau scriere.
- ▶ În funcție de accesul concurent permis, mașinile PRAM pot fi de tipul:
 - ▶ EREW (Exclusive-Read, Exclusive-Write),
 - ▶ CREW (Concurrent-Read, Exclusive-Write),
 - ▶ CRCW (Concurrent-Read, Concurrent-Write).
- ▶ O mașină EREW-PRAM nu permite acces simultan la un registru de memorie. Aceasta înseamnă că, la un moment dat, cel mult un procesor poate accesa un registru de memorie.
- ▶ Ultimele două tipuri (CREW, CRCW) permit citiri simultane din același registru de memorie. Diferențierea derivă din accesul la scriere.
- ▶ Tipul CREW-PRAM definește o mașină în care un singur procesor poate accesa la un moment dat un registru de memorie pentru scriere.
- ▶ O mașină CRCW-PRAM este caracterizată prin faptul că sunt permise accesări concurente și pentru scriere.

Modelul Message-Passing

- ▶ Un sistem Message-Passing este compus din n unități de procesare $p_0, p_1, \dots, p_{n-1}$ care comunică între ele prin mesaje transmise pe canalele de comunicare care le conectează.
- ▶ Topologia de comunicare este reprezentată printr-un graf $G = (V, E)$, care asignează fiecărui vârf v_i o unitate de procesare p_i . Fiecare muchie $e_k \in E$ reprezintă un canal de comunicare directă între două unități de procesare p_i și p_j .
- ▶ Fiecare unitate de procesare p_i poate fi într-o stare din mulțimea de stări Q_i . Fiecare stare include două mulțimi de regiștri: $(inbuf_i^k, k = 1, \dots, d)$ și $(outbuf_i^k, k = 1, \dots, d)$, unde d este gradul vârfului v_i asignat lui p_i .
- ▶ Un registru $inbuf_i^k$ conține mesajele pe care unitatea de procesare p_i le-a primit de la unitatea de procesare vecină p_j , prin canalul de comunicare directă k . Un registru $outbuf_i^k$ conține mesajele pe care unitatea de procesare p_i le-a transmis spre unitatea de procesare vecină p_j , prin canalul de comunicare directă k .
- ▶ Mulțimea de stări Q_i conține o submulțime de stări speciale, în care regiștrii $inbuf_i$ nu memorează nici un mesaj.
- ▶ Evenimentele sunt pași *compuționali* și pași de *comunicare*.
- ▶ În timpul unui pas compuțional, o unitate de procesare p_i citește și apoi resetează regiștrii $inbuf_i^k, k = 1, \dots, d$ și produce valori noi pentru $outbuf_i^k, k = 1, \dots, d$, schimbându-și în același timp starea internă.
- ▶ Evenimentul de comunicare înseamnă o operație de transmisie, i.e. un mesaj memorat în $outbuf$ -ul unei unități de procesare p_i este transmis, prin canalul de comunicare directă, la destinația p_j , unde va fi memorat în $inbuf$.

Modelul sistolic

- ▶ Este derivat din modelul Message-Passing.
- ▶ Arhitecturile utilizate de modelul sistolic au două proprietăți foarte importante: regularitatea și interconexiunile locale.
- ▶ O arhitectură sistolică este
 - ▶ compusă din celule computaționale de același tip sau dintr-o mulțime restrânsă de tipuri,
 - ▶ interconectate regulat prin conexiuni locale.



Y. Robert și D. Trystram
Problema algebrică a drumurilor

Figura 21: Exemplu de arhitectură sistolică

Cuprins

1. Introducere
2. Modele de comunicare
3. Topologii de comunicare
4. Modele de calcul paralel/distribuit
5. Parametri de performanță a unui algoritm paralel/distribuit
6. Comunicarea în sistemele de calcul paralel/distribuit
7. Algoritmi fundamentali
8. Sortare paralelă
9. Calcul matriceal
10. Sisteme de ecuații liniare
11. Alegerea liderului
12. Excluderea mutuală în sisteme *shared-memory*

Parametri de performanță a unui algoritm paralel/distribuit

- ▶ Parametri de performanță a unui algoritm paralel:
 - ▶ timpul de execuție paralelă: $T_p(n)$, p = numărul unităților de procesare, n = dimensiunea problemei;
 - ▶ timpul de execuție secvențială: $T_s(n)$;
 - ▶ factorul de încărcare: $L = n/p$;
 - ▶ accelerarea: $S(n) = T_s(n)/T_p(n)$, $T_s(n)$ = timpul de execuție al celui mai rapid algoritm secvențial;
 - ▶ eficiența: $E(n) = S(n)/p = T_s(n)/(pT_p(n))$;
 - ▶ costul: $C(n) = \text{Costul} = pT_p(n)$
- ▶ Parametri de performanță a unui algoritm distribuit:
 - ▶ numărul de mesaje transmise prin rețeaua de interconectare: $M(n)$, p = numărul unităților de procesare.

Cuprins

1. Introducere
2. Modele de comunicare
3. Topologii de comunicare
4. Modele de calcul paralel/distribuit
5. Parametri de performanță a unui algoritm paralel/distribuit
6. Comunicarea în sistemele de calcul paralel/distribuit
7. Algoritmi fundamentali
8. Sortare paralelă
9. Calcul matriceal
10. Sisteme de ecuații liniare
11. Alegerea liderului
12. Excluderea mutuală în sisteme *shared-memory*

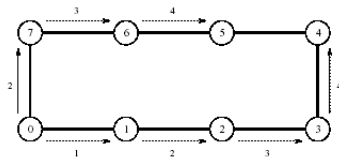
Tipuri de comunicare

- ▶ unu la unu.
- ▶ comunicări colective:
 - ▶ unu la un grup, unu-la-toți: broadcast, scatter;
 - ▶ un grup la un grup, un grup la toți, toți la un grup, toți la toți;
 - ▶ un grup la unu, toți la unu: reduction, gather.

Difuzia *unu la toți*

Definiție: O unitatea de procesare trimite același mesaj M la toate celelalte.

Difuzia *unu la toți* pe un inel



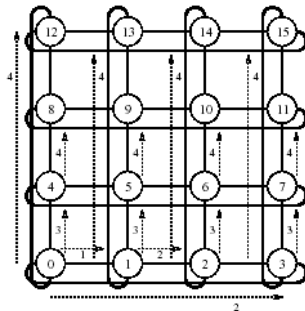
Copyright © V. Kumar, A. Grama A. Gupta & G Karypis, Introduction to Parallel Computing: Design and Analysis of Algorithms, Addison Wesley, 2003.

Figura 22: Exemplu de difuzie *unu la toți* pe un inel

- Timpul de execuție: $T = (T_s + T_c m)[p/2]$, unde
 - T_s = timpul de inițializare a transmisiei;
 - T_c = timpul de transmisie a unui cuvânt;
 - m = numărul de cuvinte care compun mesajul M ; prin cuvânt se înțelege unitatea lexicală care compune mesajul (exemplu: octet, 2 octeți, etc.);
 - p = numărul de unități de procesare.

- └ Comunicarea în sistemele de calcul paralel/distribuit
 - └ Comunicare prin rețele de interconectare cu topologie regulată

Difuzia *unu la toți* pe o plasă circulară

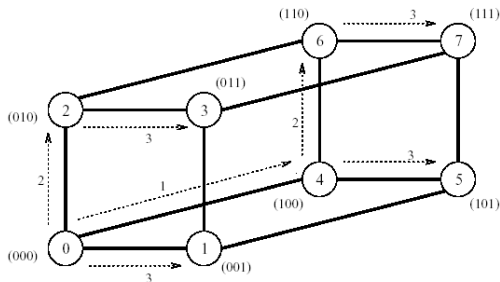


Copyright © V. Kumar, A. Grama A. Gupta & G Karypis, Introduction to Parallel Computing: Design and Analysis of Algorithms, Addison Wesley, 2003.

Figura 23: Exemplu de difuzie *unu la toți* pe o plasă circulară

$$\text{Timpul de execuție: } T = (T_s + T_c m)[\sqrt{p}].$$

Difuzia *unu la toți* pe un hipercub



Copyright © V. Kumar, A. Grama A. Gupta & G Karypis, Introduction to Parallel Computing: Design and Analysis of Algorithms, Addison Wesley, 2003.

Figura 24: Exemplu de difuzie *unu la toți* pe un hipercub

$$\text{Timpul de execuție: } T = (T_s + T_c m) \log p$$

Algoritmul de difuzie *unu la toți* pe un hipercub

► *Notatii:*

- H este un hipercub cu d dimensiuni.
- M este un mesaj care urmează a fi transmis tuturor unităților de procesare.
- i este identificatorul unității de procesare care execută algoritmul de difuzie *unu la toți* pe hipercub.

► *Premise:*

- Inițial, mesajul M se află în nodul 0 al hipercubului H .

Algoritmul de difuzie *unu la toți* pe un hipercub

Algoritm 1: Difuzia pe hipercub – rădăcină nodul 0

```

1:  $masca \leftarrow 2^d - 1$ 
2: for  $k \leftarrow d - 1 \rightarrow 0$  do
3:    $masca \leftarrow masca \text{ XOR } 2^k$ 
4:   if  $i \text{ AND } masca = 0$  then
5:     if  $i \text{ AND } 2^k = 0$  then
6:        $destinatie \leftarrow i \text{ XOR } 2^k$ 
7:       trimite  $M$  la  $destinatie$ 
8:     else
9:        $sursa \leftarrow i \text{ XOR } 2^k$ 
10:      primește  $M$  la  $sursa$ 

```

▷ inițializează cu 1 toți biții măștii

▷ setează pe 0 al k -lea bit al măștii
 ▷ dacă ultimii k biți sunt 0

Algoritmul generalizat de difuzie *unu la toți* pe un hipercub

► *Notatii:*

- H este un hipercub cu d dimensiuni.
- M este un mesaj care urmează a fi transmis tuturor unităților de procesare.
- i este identificatorul unității de procesare care execută algoritmul de difuzie *unu la toți* pe hipercub.

► *Premise:*

- Inițial, mesajul M se află în nodul s al hipercubului H .

Algoritmul generalizat de difuzie *unu la toți* pe un hipercub

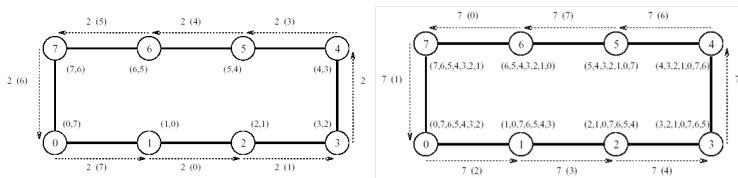
Algoritm 2: Difuzia pe hipercub – rădăcină nodul s

```
1:  $id\_virtual \leftarrow i \text{ XOR } s$   
2:  $masca \leftarrow 2^d - 1$  ▷ inițializează cu 1 toți biții măștii  
3: for  $k \leftarrow d - 1 \rightarrow 0$  do  
4:    $masca \leftarrow masca \text{ XOR } 2^k$  ▷ setează pe 0 al  $k$ -lea bit al măștii  
5:   if  $id\_virtual \text{ AND } masca = 0$  then ▷ dacă ultimii  $k$  biți sunt 0  
6:     if  $id\_virtual \text{ AND } 2^k = 0$  then  
7:        $destinatie\_virtuala \leftarrow id\_virtual \text{ XOR } 2^k$   
8:       trimite  $M$  la  $destinatie\_virtuala \text{ XOR } s$   
9:     else  
10:       $sursa\_virtuala \leftarrow id\_virtual \text{ XOR } 2^k$   
11:      primește  $M$  la  $sursa\_virtuala \text{ XOR } s$ 
```

Difuzia *toți la toți*

Definiție: Fiecare unitatea de procesare i trimite un mesaj M_i la toate celelalte.

Difuzia *toți la toți* pe un inel



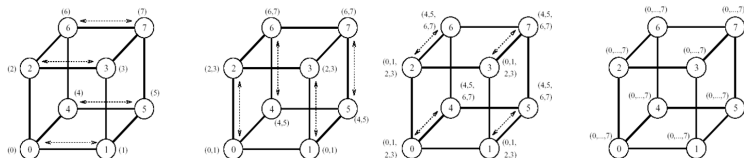
Copyright © V. Kumar, A. Grama, A. Gupta & G Karypis, Introduction to Parallel Computing: Design and Analysis of Algorithms, Addison Wesley, 2003.

Figura 25: Exemplu de difuzie *toți la toți* pe un inel – pașii 2 și 7

$$\text{Timpul de execuție: } T = (T_s + T_c m)(p - 1)$$

- └ Comunicarea în sistemele de calcul paralel/distribuit
 - └ Comunicare prin rețele de interconectare cu topologie regulată

Difuzia *toți la toți* pe un hipercub



Copyright © V. Kumar, A. Grama, A. Gupta & G Karypis, Introduction to Parallel Computing: Design and Analysis of Algorithms, Addison Wesley, 2003.

Figura 26: Exemplu de difuzie *toți la toți* pe un hipercub

$$\text{Timpul de execuție: } T = T_s \log p + T_c m(p-1)$$

Algoritmul de difuzie *toți la toți* pe un hipercub

► *Notatii:*

- H este un hipercub cu d dimensiuni.
- i este identificatorul unității de procesare care execută algoritmul de difuzie *unu la toți* pe hipercub.
- M_i este mesajul care urmează a fi transmis de unitatea de procesare i tuturor unităților de procesare.

► *Premise:*

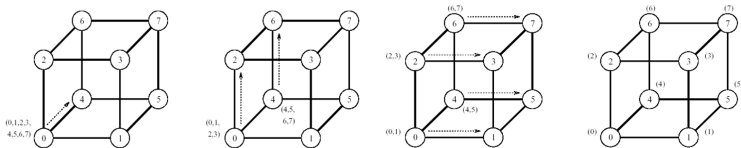
- Inițial, fiecare unitate de procesare i deține un mesaj M_i care urmează a fi transmis tuturor unităților de procesare plasate în nodurile hipercubului H .

Algoritm 3: Difuzia *toți la toți* pe hipercub

```
1:  $mesaje\_colectate \leftarrow M_i$ 
2: for  $k \leftarrow 0 \rightarrow d - 1$  do
3:    $partener \leftarrow i \text{ XOR } 2^k$ 
4:   trimite  $mesaje\_colectate$  la  $partener$ 
5:   primește  $mesaje\_partener$  de la  $partener$ 
6:    $mesaje\_colectate \leftarrow mesaje\_colectate \cup mesaje\_partener$ 
```

Comunicarea personalizată *unu la toți*

Definiție: O unitate de procesare i trimite un trimite câte un mesaj M_{ij} la fiecare unitate de procesare j .



Copyright © V. Kumar, A. Grama, A. Gupta & G Karypis, Introduction to Parallel Computing: Design and Analysis of Algorithms, Addison Wesley, 2003.

Figura 27: Exemplu de comunicare personalizată *unu la toți* pe un hipercub

$$\text{Timpul de execuție: } T = T_s \log p + T_c m(p-1)$$

Comunicarea personalizată *toți la toți*

Definiție: Fiecare unitate de procesare i trimite câte un trimite un mesaj M_{ij} la fiecare unitate de procesare j .

Difuzia *unu la toți* pe un arbore

Premise: Inițial, unitatea de procesare p_r (rădăcina) deține mesajul M , care urmează a fi difuzat în arbore.

Algoritm 4: Difuzia *unu la toți* pe arbore oarecare – proces rădăcină

- 1: trimite M către fii
 - 2: termina executia
-

Algoritm 5: Difuzia *unu la toți* pe arbore oarecare – proces intern/frunză

- 1: asteapta primirea mesajului M de la parinte
 - 2: trimite M către fii
 - 3: termina executia
-

Complexitatea

Teorema (1)

Numărul de pași, după care toate unitățile de procesare termină execuția algoritmului = h (adâncimea arborelui T).

Demonstrație.

Inducție.



Teorema (2)

Numărul de mesaje, care sunt transmise de toate unitățile de procesare, în timpul execuției algoritmului = $n - 1$.

Demonstrație.

Pe fiecare muchie este transmis câte un mesaj, de la părinte spre fii.



Comunicare de tip *toți la unu* pe un arbore

Premise: Inițial, unitatea de procesare din frunza i deține mesajul M_i , care urmează a fi transmis spre rădăcină.

Algoritm 6: Comunicare *toți la unu* pe arbore oarecare – proces frunză

- 1: trimite M catre parinte
 - 2: termina executia
-

Algoritm 7: Comunicare *toți la unu* pe arbore oarecare – proces intern

- 1: asteapta primirea valorilor $M_{i_0}, M_{i_1}, \dots, M_{i_{k-1}}$ de la fii
 - 2: calculeaza $M_i \leftarrow f(M_{i_0}, M_{i_1}, \dots, M_{i_{k-1}})$
 - 3: trimite M_i catre parinte
 - 4: termina executia
-

Algoritm 8: Comunicare *unu la toți* pe arbore oarecare – proces rădăcină

- 1: asteapta primirea valorilor $M_{r_0}, M_{r_1}, \dots, M_{r_{k-1}}$ de la fii
 - 2: calculeaza $M_r \leftarrow f(M_{r_0}, M_{r_1}, \dots, M_{r_{k-1}})$
 - 3: termina executia
-

Complexitatea

Teorema (3)

Numărul de pași, după care toate unitățile de procesare termină execuția algoritmului = h (adâncimea arborelui T).

Demonstrație.

Inducție.



Teorema (4)

Numărul de mesaje, care sunt transmise de toate unitățile de procesare, în timpul execuției algoritmului = $n - 1$.

Demonstrație.

Pe fiecare muchie este transmis câte un mesaj, de la fiu către părinte.



Comunicare *grup la grup*

- ▶ Comunicarea între grupuri este similară cu comunicarea între toate unitățile de procesare ale sistemului.
- ▶ Deosebiri sunt următoarele:
 - ▶ Pe parcursul execuției algoritmului de comunicare doar unitățile de procesare membre ale grupurilor distincte vor iniția un transfer sau vor memora un mesaj.
 - ▶ Celelalte unități de procesare au rol de retransmisie a mesajelor (releu).

Cuprins

1. Introducere
2. Modele de comunicare
3. Topologii de comunicare
4. Modele de calcul paralel/distribuit
5. Parametri de performanță a unui algoritm paralel/distribuit
6. Comunicarea în sistemele de calcul paralel/distribuit
7. Algoritmi fundamentali
8. Sortare paralelă
9. Calcul matriceal
10. Sisteme de ecuații liniare
11. Alegerea liderului
12. Excluderea mutuală în sisteme *shared-memory*

Comprimarea (Reducerea)

- ▶ Fie M o mulțime de $n = 2^m$ elemente, $M = \{a_i | i = 0, 1, \dots, n-1\} \subseteq M_r =$ mulțime de referință.
- ▶ Mulțimea M urmează a fi procesată pentru a calcula valoarea $a_1 \oplus \dots \oplus a_n$, unde $\oplus$ este o lege de compoziție asociativă definită pe M_r .
- ▶ Mulțimea de referință M_r poate fi $\mathbb{R}$, iar $\oplus$ poate fi $+$, $\min$, $\max$, etc.

Algoritm recursiv de comprimare

► *Notatii:*

- $A[0..n-1]$ este un tablou de dimensiune $n = 2^m$.
- Prin $(A, k, k+l-1)$ se notează segmentul din tabloul A care începe pe poziția k și se termină pe poziția $k+l-1$

► *Premise:*

- Datele de intrare sunt inițial memorate în tabloul $A[0..n-1]$.
- Inițial, $k = 0$ și $l = n$.

Algoritm 9: Algoritmul de comprimare – versiunea recursivă

```
1: procedure COMPRIM_RECURSIV( $A, k, k+l-1$ )
2:   if  $l = 1$  then
3:     return  $A[k]$ 
4:   else
5:     return  $\text{COMPRIM\_RECURSIV}(A, k, k+l/2-1) \oplus$ 
               $\text{COMPRIM\_RECURSIV}(A, k+l/2, k+l-1)$ 
```

Algoritm iterativ de comprimare

► *Notatii:*

- $A[0..2n-1]$ este un tablou de dimensiune $2n = 2^{m+1}$.

► *Premise:*

- Datele de intrare sunt memorate în tabloul $A[0..2n-1]$, în locațiile $A[n], A[n+1], \dots, A[2n-1]$.

Algoritm 10: Algoritmul de comprimare – versiunea iterativă

```
1: procedure COMPRIM_ITER( $A, \oplus$ )
2:   for  $k \leftarrow m-1 \rightarrow 0$  do
3:     for  $j \leftarrow 2^k \rightarrow 2^{k+1}-1$  do
4:        $A[j] \leftarrow A[2j] \oplus A[2j+1]$ 
```

Algoritm paralel de comprimare

► *Notatii:*

- $A[0..2n-1]$ este un tablou de dimensiune $2n = 2^{m+1}$.

► *Premise:*

- Datele de intrare sunt memorate în tabloul $A[0..2n-1]$, în locațiile $A[n], A[n+1], \dots, A[2n-1]$.

Algoritm 11: Algoritmul de calcul paralel pentru comprimare

```
1: procedure COMPRIM_PARALEL( $A, \oplus$ )
2:   for  $k \leftarrow m-1 \rightarrow 0$  do
3:     for all  $j: 2^k \leq j \leq 2^{k+1} - 1$  parallel do
4:        $A[j] \leftarrow A[2j] \oplus A[2j+1]$ 
5:     end for
```

Exemplu de execuție a algoritmului de comprimare

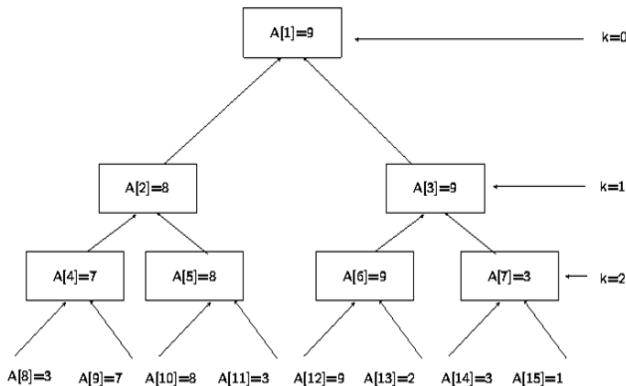


Figura 28: Exemplu de execuție a algoritmului de comprimare

Corectitudinea și complexitatea

Teorema (1)

La terminarea execuției algoritmului 11 $A[1] = a_0 \oplus a_1 \oplus \dots a_{n-1}$.

Teorema (2)

Complexitatea timp a algoritmului de comprimare (implementat pe o mașină CREW-PRAM sau pe o arhitectură VLSI de tip arbore binar) este $O(\log n)$.

Demonstrație.

Adâncimea arborelui de calcul este $\log n$.



Teorema (3)

Numărul unităților de procesare este $p \geq \lceil \frac{n}{2} \rceil$

Demonstrație.

În prima iterație sunt active $\lceil \frac{n}{2} \rceil$ unități de procesare. Apoi numărul lor scade pâna la 1.



Reducerea numărului unităților de procesare

- ▶ Se poate observa că în fiecare pas $k, k = m-1, m-2, \dots, 0$, sunt active $\lceil \frac{n}{2^{m-k}} \rceil$ unități de procesare. Aceasta sugerează faptul că este posibilă reducerea numărului de unități de procesare, fără a afecta complexitatea algoritmului.
- ▶ Presupunând că $p < \lceil \frac{n}{2} \rceil$, se partiționează mulțimea celor n elemente $a_0, a_1, \dots, a_{n-1}$ în p submulțimi astfel:
 - ▶ Primele $p-1$ submulțimi contin $k = \lceil \frac{n}{p} \rceil$ elemente.
 - ▶ Ultima submulțime conține $n - (p-1) \cdot \lceil \frac{n}{p} \rceil$ ($\leq \lceil \frac{n}{p} \rceil$) elemente.
- ▶ Fiecărei submulțimi i se asociază o unitate de procesare. Unitatea de procesare asociată unei submulțimi $S = \{a_{i_0}, \dots, a_{i_{k-1}}\}$ rezolvă problema determinării valorii $a_{i_0} \oplus \dots \oplus a_{i_{k-1}}$ în $\lceil \frac{n}{p} \rceil - 1$ unități de timp.
- ▶ Actionând în paralel, cele p unități de procesare reduc problema inițială la o problemă similară de dimensiune p , în $\lceil \frac{n}{p} \rceil - 1$ unități de timp.
- ▶ În continuare, aplicând comprimarea asupra problemei rezultate, se obține rezultatul final în $\log p$ unități de timp.
- ▶ Rezultă un timp total de $\lceil \frac{n}{p} \rceil - 1 + \log p$ unități. Dacă $p = \lceil \frac{n}{\log n} \rceil$, atunci $\lceil \frac{n}{p} \rceil - 1 + \log p = 2 \log n - 1 - \log \log n \in O(\log n)$,

Comentarii

- ▶ Comprimarea nu se reduce la operarea cu elementele unei mulțimi de numere.
- ▶ $a_0 \oplus a_1 \oplus \dots \oplus a_{n-1}$ pot fi n fișiere, care urmează a fi concatenate.
- ▶ Mai pot fi n instanțe ale unei tabele ale unei baze de date distribuite, care trebuie unificate.
- ▶ În aceste situații algoritmul este distribuit.

Calculul Prefixelor (Scan)

- ▶ Tehnica de calcul a prefixelor dezvoltă tehnica de comprimare, în care parcurgerea arborelui asociat execuției algoritmului se face dinspre frunze spre rădăcină.
- ▶ Algoritmii de calcul a prefixelor parcurg arborele de execuție în ambele sensuri.
- ▶ Calculul prefixelor constă în determinarea valorilor $a_0, a_0 \oplus a_1, \dots, a_0 \oplus a_1 \oplus \dots \oplus a_{n-1}$, unde $\oplus$ este o operație binară asociativă.

Algoritmul de calcul al prefixelor în cazul în care operația $\oplus$ admite element simetric

► *Notatii:*

- $A[0..2n-1]$ este un tablou de dimensiune $2n = 2^{m+1}$.

► *Premise:*

- Datele de intrare sunt memorate în tabloul $A[0..n-1]$, în locațiile $A[n], A[n+1], \dots, A[2n-1]$.

Algoritm 12: Algoritmul paralel pentru calculul prefixelor

```

1: procedure CALCUL_PREFIXE_SIMETRIC( $A, \oplus$ )
2:   COMPRIM_PARALEL( $A, \oplus$ )
3:   for  $k \leftarrow 1 \rightarrow m$  do
4:     for all  $j: 2^k \leq j \leq 2^{k+1} - 1$  parallel do
5:       if  $\text{bit}_0(j) = 1$  then
6:          $B[j] \leftarrow B[(j-1)/2]$ 
7:       else
8:          $B[j] \leftarrow B[j/2] \oplus (-A[j+1])$ 
9:     end for
```

► Algoritmul 11

Exemplu de execuție a algoritmului de calcul al prefixelor în cazul în care operația $\oplus$ admite element simetric

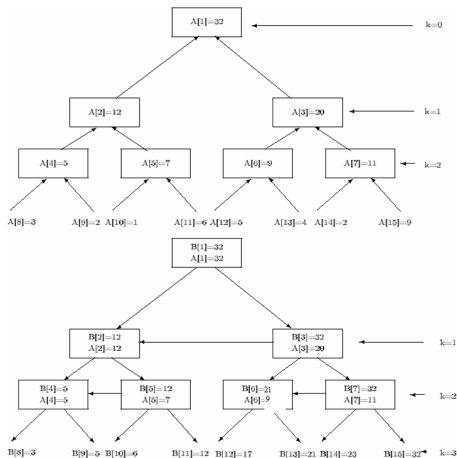


Figura 29: Exemplu de execuție a Algoritmului 12 – operația $\oplus$ admite element simetric

Algoritmul de calcul al prefixelor în cazul în care operația $\oplus$ nu admite element simetric

- *Notății:* $A[0..2n-1]$ și $B[0..2n-1]$ sunt tablouri de dimensiune $2n = 2^{m+1}$.
- *Premise:* Datele de intrare sunt memorate în tabloul $A[0..2n-1]$, în locațiile $A[n], A[n+1], \dots, A[2n-1]$. La final,
 $B[n] = a_0, B[n+1] = a_0 \oplus a_1, \dots, B[2n-1] = a_0 \oplus a_1 \oplus \dots \oplus a_{n-1}$.

Algoritm 13: Algoritmul paralel pentru calculul prefixelor ($\oplus$ nu admite element simetric)

```
1: procedure CALCUL_PREFIXE_NESIMETRIC( $A, \oplus$ )  
2:   COMPRIM_PARALEL( $A, \oplus$ )  
3:   for  $k \leftarrow 1 \rightarrow m$  do  
4:     for all  $j: 2^k \leq j \leq 2^{k+1} - 1$  parallel do  
5:       if  $\text{bit}_0(j) = 1$  then  
6:          $B[j] \leftarrow B[(j-1)/2]$   
7:       else  
8:          $B[j] \leftarrow B[(j-2)/2] \oplus (A[j])$   
9:     end for
```

▷ Algoritmul 11

Corectitudinea și complexitatea

Teorema (4)

La terminarea execuției algoritmilor 12 și 13,

$$B[n] = a_0, \quad B[n+1] = a_0 \oplus a_1, \dots, B[2n-1] = a_0 \oplus a_1 \oplus \dots \oplus a_{n-1}.$$

Teorema (5)

Complexitatea timp a algoritmului paralel de calcul al prefixelor (implementat pe o masina CREW-PRAM sau pe o arhitectura VLSI de tip arbore binar) este $O(\log n)$.

Demonstrație.

Adâncimea arborelui de calcul este $\log n$.



Teorema (6)

Numărul unităților de procesare este $\lceil \frac{n}{\log n} \rceil$.

Demonstrație.

Necesarul de unități de procesare este același cu cel de la comprimare.



Comentarii

- ▶ Similar comprimării, calculul prefixelor nu se reduce la operarea cu elementele unei mulțimi de numere.
- ▶ Compilatoarele folosesc masiv calculul prefixelor.
- ▶ Descoperirea tiparelor frecvente în colecții de date de dimensiuni foarte mari, conține ca task de bază calculul de prefixe.
- ▶ Regăsirea informației este un alt domeniu de aplicare a calculului prefixelor.
- ▶ Algoritmii paraleli pot aduce performanță sporită.
- ▶ Recurențele sunt cel mai clar exemplu de aplicare a calculului prefixelor.
 - ▶ Fie recurența $x_k = x_{k-1} \oplus a_k, k > 0, x_0 = a_0$.
 - ▶ Dacă $\oplus$ este un operator binar asociativ, atunci $x_k = a_0 \oplus a_1 \oplus \dots \oplus a_k$.

Scurtcircuitarea (dublarea)

- ▶ Fie L o listă simplu înlănțuită formată din n elemente.
- ▶ Fiecare element $k \in L$ are asociat un număr $valoare[k]$ și un pointer $pointer[k]$.
- ▶ Se presupune că fiecărui element $k \in L$ îi este asignată o unitate de procesare p_k .
- ▶ Scurtcircuitarea se referă la indirectarea prin intermediul pointerilor, pentru a accesa locații noi.

Algoritm paralel de scurtcircuitare

Algoritm 14: Algoritmul paralel de scurtcircuitare

```
1: procedure SCURTCIRCUITARE_PARALEL( $L$ ,  $valoare$ ,  $\oplus$ )
2:   for  $i \leftarrow 1 \rightarrow \log n$  do
3:     for all  $k: k \in L$  parallel do
4:       if  $pointer[k] \neq pointer[pointer[k]]$  then
5:          $valoare[k] \leftarrow valoare[k] \oplus valoare[pointer[k]]$ 
6:          $pointer[k] \leftarrow pointer[pointer[k]]$ 
7:     end for
```

Complexitatea

Teorema (7)

Complexitatea timp a algoritmului paralel de scurtcircuitare, implementat pe o masina CREW-PRAM, este $O(\log n)$.

Aplicații ale scurtcircuitării: Calculul rangurilor

- ▶ Un exemplu tipic de utilizare a tehnicii scurtcircuitării îl constituie calcularea numerelor de ordine ale elementelor unei liste, față de sfârșitul acesteia.
- ▶ Pentru un element $k \in L$, $\text{rang}[k]$ va conține în final numărul i al elementor din lista L care se afla în listă după elementul k , în ordinea dată de pointeri.

Algoritm paralel pentru calculul rangurilor

Algoritm 15: Algoritm paralel pentru calculul rangurilor

```
1: procedure CALCUL_RANGURI_PARALEL( $L$ ,  $\text{valoare}$ ,  $+$ )
2:   for all  $k$ :  $k \in L$  parallel do
3:      $\text{pointer}[k] \leftarrow \text{succesor}[k]$ 
4:     if  $\text{pointer}[k] \neq k$  then
5:        $\text{rang}[k] \leftarrow 1$ 
6:     else
7:        $\text{rang}[k] \leftarrow 0$ 
8:   end for
9:   SCURTCIRCUITARE_PARALEL( $L$ ,  $\text{rang}$ ,  $+$ )
```

▷ Algoritmul 14

Exemplu de execuție a algoritmului de calcul al rangurilor

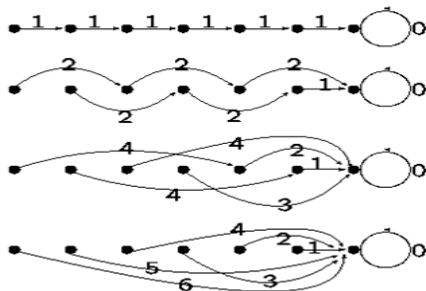


Figura 30: Exemplu de execuție a algoritmului de calcul ranguri al rangurilor

Comentarii

- ▶ Scurtcircuitarea nu se reduce la operarea cu elementele unei liste.
- ▶ Documentele Web pot fi obiectul unor operații de scurtcircuitare paralelă, atunci când se dorește clusterizarea acestora.
- ▶ Rutarea poate folosi tehnica scurtcircuitării paralele.

Cuprins

1. Introducere
2. Modele de comunicare
3. Topologii de comunicare
4. Modele de calcul paralel/distribuit
5. Parametri de performanță a unui algoritm paralel/distribuit
6. Comunicarea în sistemele de calcul paralel/distribuit
7. Algoritmi fundamentali
- 8. Sortare paralelă**
9. Calcul matriceal
10. Sisteme de ecuații liniare
11. Alegerea liderului
12. Excluderea mutuală în sisteme *shared-memory*

Sortare paralelă

- ▶ Există o vastă literatură de specialitate având ca subiect sortarea.
- ▶ Aceasta se explică prin faptul că sortarea apare ca subtask în soluțiile algoritmice ale multor probleme.
- ▶ Problema poate fi enunțată astfel:
Date fiind n elemente $a_0, a_1, \dots, a_{n-1}$, dintr-o mulțime U peste care este definită o relație de ordine totală " $<$ ", se dorește renumerotarea lor astfel încât $a_i < a_j$, $i, j \in \{0, 1, \dots, n-1\}$, $i < j$.
- ▶ Se presupune, pentru simplitate, că $a_i \neq a_j$, dacă $i \neq j$.

Algoritmul Muller-Preparata - descriere

- ▶ Autori: David E. Muller și Franco P. Preparata. Anul publicării: 1975
- ▶ Algoritmul este compus din trei faze:
 1. Determinarea pozițiilor relative pentru fiecare pereche $\{a_i, a_j\}, i, j = 0, 1, \dots, n-1$:
 - ▶ Notății: $pozitie_relativa_{a_j}(a_i)$ și $pozitie_relativa_{a_i}(a_j)$ desemnează poziția lui a_i , respectiv a_j în secvența (a_i, a_j) sortată crescător.
 - ▶ Dacă $a_i < a_j$, atunci $pozitie_relativa_{a_j}(a_i) = 0$ și $pozitie_relativa_{a_i}(a_j) = 1$.
 - ▶ Dacă $a_i > a_j$, atunci $pozitie_relativa_{a_j}(a_i) = 1$ și $pozitie_relativa_{a_i}(a_j) = 0$.
 2. Calcularea pozițiilor finale ale elementelor a_i , $i = 0, 1, \dots, n-1$.
 3. Plasarea elementelor a_j pe pozițiile finale.

Algoritmul Muller-Preparata – pseudocod

► *Notatii:*

- $A[0..n-1]$ și $P[0..n-1]$ sunt două tablouri, fiecare de dimensiune n .
- $R[0..2n-2, 0..n-1]$ este un tablou bidimensional de mărime $(2n-1) \times n$; $R[j]$ desemnează coloana j .

► *Premise:*

- Datele de intrare sunt memorate în tabloul $A[0..n-1]$.
- Pozițiile relative vor fi memorate în tabloul R , în liniile $n-1, n, \dots, 2n-2$.
- Pozițiile finale vor fi reținute în tabloul P .

Algoritmul Muller-Preparata – pseudocod

Algoritm 16: Algoritmul Muller-Preparata

```
1: procedure SORTARE_PARALELA_MULLER_PREPARATA( $A, R, n$ )
2:   for all  $i, j$ :  $0 \leq i, j \leq n-1$  parallel do
3:     if  $A[i] < A[j]$  then
4:        $R[i + n - 1, j] \leftarrow 1$ 
5:     else
6:        $R[i + n - 1, j] \leftarrow 0$ 
7:   end for
8:   for all  $j$ :  $0 \leq j \leq n-1$  parallel do
9:     COMPRIM_ITER( $R[j], +$ )
10:     $P[j] \leftarrow R[0, j]$ 
11:  end for
12:  for all  $j$ :  $0 \leq j \leq n-1$  parallel do
13:     $A[P[j]] \leftarrow A[j]$ 
14:  end for
```

▷ Algoritmul 10

Exemplu de execuție a algoritmului Muller-Preparata

Tabloul R

a_j j i		2	6	3	8
		0	1	2	3
0		0	2	1	3
1		0	1	1	2
2	a_i	0	1	0	1
3		0	1	1	1
4		0	0	0	1
5		0	1	0	1
6		0	0	0	0

Figura 31: Exemplu de execuție a algoritmului de sortare paralelă Muller-Preparata pentru secvența 2,6,3,8

Lema (1)

Pozițiile finale ale elementelor secvenței $a_0, a_1, \dots, a_{n-1}$ respectă relația de ordine " $<$ ".

Demonstrație.

În urma calculării numărului elementelor care se află în fața unui element a_i (COMPRIM_ITER($R[j]$, +)), se obține poziția finală a acestuia, în concordanță cu relația de ordine " $<$ ". □

Teorema (1)

Algoritmul Muller-Preparata sortează corect o secvență de elemente $a_0, a_1, \dots, a_{n-1}$ dintr-o mulțime U peste care este definită o relație de ordine totală " $<$ ".

Demonstrație.

Consecință imediată a lemei 1. □

Complexitatea

Teorema (2)

Complexitatea timp a algoritmului de sortare paralelă Muller-Preparata, implementat pe o masina CREW-PRAM cu $O(\frac{n^2}{\log n})$ unități de procesare, este $O(\log n)$.

Demonstrație.

1. Determinarea pozițiilor relative pentru fiecare pereche $\{a_i, a_j\}, i, j = 0, 1, \dots, n-1$: dacă mașina CREW-PRAM este compusă din n^2 unități de procesare, timpul paralel este $O(1)$; dacă numărul unităților de procesare este $\lceil \frac{n^2}{\log n} \rceil$, timpul paralel este $O(\log n)$ (tehnica este aceeași cu cea de la comprimare).
2. Calcularea pozițiilor finale ale elementelor $a_i, i = 0, 1, \dots, n-1$: pentru fiecare i , sunt necesare cel puțin $\lceil \frac{n}{\log n} \rceil$ unități de procesare, pentru a calcula poziția finală a elementului a_i în timpul paralel $O(\log n)$ (vezi complexitatea algoritmului paralel de comprimare). Rezultă un necesar de $n \lceil \frac{n}{\log n} \rceil$ unități de procesare pentru a calcula toate pozițiile finale în timpul paralel $O(\log n)$.
3. Plasarea elementelor a_j pe pozițiile corecte necesită n unități de procesare pentru se obține timpul paralel $O(1)$.



Comentarii

- ▶ Relativ la algoritmul secvențial, bazat pe metoda enumerării, care necesită $O(n^2)$ timp, eficiența algoritmului este $E = \frac{O(n^2)}{O(\frac{n^2}{\log n}) \log n} = O(1)$.
- ▶ Totuși, algoritmul nu este optimal, deoarece cel mai rapid algoritm secvențial are timpul de execuție de $O(n \log n)$.

Algoritmul Impar-Par (Odd-Even Sort) - descriere

- ▶ Este o versiune a algoritmului Bubble Sort.
- ▶ Se desfășoară în faze.
 - ▶ În fazele impare sunt sortate perechile $\{a_i, a_{i+1}\}$ cu i par.
 - ▶ În fazele pare sunt sortate perechile $\{a_i, a_{i+1}\}$ cu i impar.
- ▶ Este paralelizabil.

Algoritmul Impar-Par secvențial – pseudocod

- *Notatii:* $A[0..n-1]$ este un tablou de dimensiune n .
- *Premise:* Datele de intrare sunt memorate în tabloul $A[0..n-1]$.

Algoritm 17: Algoritmul Impar-Par secvențial

```
1: procedure IMPAR_PAR_SEQUENTIAL( $A, n$ )
2:   for faza  $i \leftarrow 1 \rightarrow n$  do
3:     if faza este impara then
4:       for  $i \leftarrow 0 \rightarrow 2\lceil \frac{n}{2} \rceil - 2$ , step 2 do
5:         CMPSWAP( $i, i+1$ )
6:     if faza este para then
7:       for  $i \leftarrow 1 \rightarrow 2\lceil \frac{n-1}{2} \rceil - 1$ , step 2 do
8:         CMPSWAP( $i, i+1$ )
```

Algoritm 18: Procedura de interschimbare

```
1: procedure CMPSWAP( $i, j$ )
2:   if  $a_i > a_j$  then
3:      $temp \leftarrow a_j$ 
4:      $a_j \leftarrow a_i$ 
5:      $a_i \leftarrow temp$ 
```

Exemplu de execuție a algoritmului Impar-Par

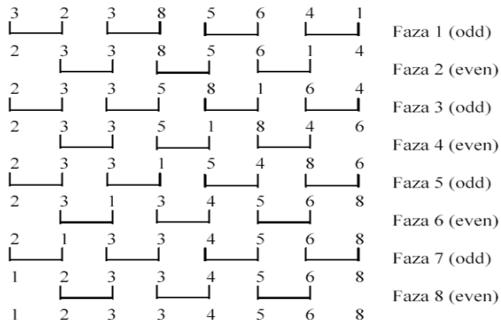


Figura 32: Exemplu de execuție a algoritmului de sortare Impar-Par pentru $n = 8$

Algoritmul Impar-Par paralel – pseudocod pentru un lanț de unități de procesare

- *Premise:* Inițial, o unitate de procesare p_i memorează elementul a_i în registrul r .

Algoritm 19: Algoritmul Impar-Par paralel

```
1: procedure IMPAR_PAR_PARALEL( $p_i, r, n$ )
2:   for faza  $i \leftarrow 1 \rightarrow n$  do
3:     if faza este impara si  $0 \leq i \leq 2\lceil \frac{n}{2} \rceil - 1$  then
4:       if  $i$  este par then
5:         trimite lui  $p_{i+1}$  valoarea memorata in  $r$ ; primește de la  $p_{i+1}$  o valoare  $v$ 
6:          $r \leftarrow \min(r, v)$ 
7:       else
8:         trimite lui  $p_{i-1}$  valoarea memorata in  $r$ ; primește de la  $p_{i-1}$  o valoare  $v$ 
9:          $r \leftarrow \max(r, v)$ 
10:    if faza este para si  $1 \leq i \leq 2\lceil \frac{n-1}{2} \rceil$  then
11:      if  $i$  este impar then
12:        trimite lui  $p_{i+1}$  valoarea memorata in  $r$ ; primește de la  $p_{i+1}$  o valoare  $v$ 
13:         $r \leftarrow \min(r, v)$ 
14:      else
15:        trimite lui  $p_{i-1}$  valoarea memorata in  $r$ ; primește de la  $p_{i-1}$  o valoare  $v$ 
16:         $r \leftarrow \max(r, v)$ 
```

Complexitatea

Teorema (3)

Complexitatea timp a algoritmului de sortare paralelă Impar-Par, implementat pe un lanț de n unități de procesare, este $O(n)$.

Demonstrație.

Timpul paralel pentru fiecare fază este $O(1)$. După n faze algoritmul se termină.



Comentarii

- ▶ Algoritmul de sortare paralelă *Impar-Par* este optimal pentru arhitectură: fiecare unitate de procesare este solicitată $O(n)$ timp
- ▶ Costul nu este optimal:
(numărul de unități de procesare) $\times$ (timpul paralel) $= n \times n = O(n^2)$.
Timpul pentru cel mai rapid algoritm secvențial este $O(n \log n)$.
- ▶ Algoritmul poate fi implementat și pe o mașină CREW-PRAM.

Exercițiu

Scrieți un pseudocod pentru sortarea Impar-Par pe o astfel de mașină

Algoritmul lui Batcher de sortare bitonică – descriere

- ▶ Autor: Batcher; Anul publicării: 1968.
- ▶ Operația de bază este sortarea unei secvențe bitone.
- ▶ Esența problemei sortării unei secvențe bitone este transformarea sortării unei secvențe de bitone lungime n în sortarea a doua secvențe bitone de dimensiune $\frac{n}{2}$.
- ▶ Pentru a sorta o secvență de n elemente, prin tehnica sortării unei secvențe bitone, trebuie să dispunem de o secvență bitonă formată din n elemente.
- ▶ Observații:
 - ▶ Două elemente formează o secvență bitonă.
 - ▶ Orice secvență nesortată este o concatenare de secvențe bitone de lungime 2.
- ▶ Ideea transformării unei secvențe oarecare în una bitonă: combinarea a două secvențe bitone de lungime $\frac{n}{2}$ pentru a obține o secvență bitonă de lungime n .
- ▶ Algoritmul este paralelizabil.

Secvențe bitone

- ▶ Secvența bitonă este o secvență de elemente $[a_0, a_1, \dots, a_{n-1}]$ pentru care
 - ▶ există i astfel încât $[a_0, a_1, \dots, a_i]$ este monoton crescătoare și $[a_{i+1}, \dots, a_{n-1}]$ este monoton descrescătoare sau
 - ▶ există o permutare circulară astfel încât să fie satisfăcută condiția anterioară.
- ▶ Exemple:
 - ▶ $[1, 2, 4, 7, 6, 0]$; întâi crește și apoi descrește; $i = 3$.
 - ▶ $[8, 9, 2, 1, 0, 4]$: după o permutare circulară la stânga cu 4 poziții rezultă $[0, 4, 8, 9, 2, 1]$; $i = 3$.
- ▶ Fie $S = [a_0, a_1, \dots, a_{n-1}]$ o secvență bitonă,
 - ▶ $S_1 = [\min\{a_0, a_{\frac{n}{2}}\}, \min\{a_1, a_{\frac{n}{2}+1}\}, \dots, \min\{a_{\frac{n}{2}-1}, a_{n-1}\}]$ și
 - ▶ $S_2 = [\max\{a_0, a_{\frac{n}{2}}\}, \max\{a_1, a_{\frac{n}{2}+1}\}, \dots, \max\{a_{\frac{n}{2}-1}, a_{n-1}\}]$
- ▶ Secvențele S_1 și S_2 au proprietățile următoare:
 - ▶ Sunt bitone.
 - ▶ Fiecare element din S_1 este mai mic decât fiecare element din S_2 .

Algoritmul lui Batcher de sortare bitonică – pseudocod

► *Notății:*

- $A[0..n-1]$ este un tablou unidimensional de dimensiune n .
 - (A, i, d) definește segmentul $A[i..i+d-1] = A[i], \dots, A[i+d-1]$.
 - s este un parametru binar care specifică ordinea crescătoare ($s = 0$) sau descrescătoare ($s = 1$) a cheilor de sortare.
 - $\text{COMPARA_SI_SCHIMBA}(x, y, s)$ desemnează sortarea a două elemente x și y ordinea indicată de parametrul s .
- *Premise:* Inițial, $A[0..n-1]$ conține secvența de sortat.
- *Apel:* $\text{SORTARE_BATCHER}(A, 0, n, 0)$ sau $\text{SORTARE_BATCHER}(A, 0, n, 1)$.

Algoritmul lui Batcher de sortare bitonică – pseudocod

Algoritm 20: Algoritmul lui Batcher de sortare bitonică

```
1: procedure SORTARE_BATCHER( $A, i, d, s$ )
2:   if  $d = 2$  then
3:      $(A[i], A[i+1]) \leftarrow \text{COMPARA\_SI\_SCHIMBA}(A[i], A[i+1], s)$ 
4:   else
5:     ▷ sortare crescătoare a unei secvențe  $S$  de lungime  $\frac{d}{2}$ 
6:     SORTARE_BATCHER( $A, i, \frac{d}{2}, 0$ )
7:     ▷ sortare descrescătoare a unei secvențe  $S'$  de lungime  $\frac{d}{2}$ 
8:     SORTARE_BATCHER( $A, i + \frac{d}{2}, \frac{d}{2}, 1$ )
9:     ▷ sortarea secvenței bitone  $SS'$ , de lungime  $d$  (Algoritmul 21)
10:    SORTARE_SECVENTA_BITONA( $A, i, d, s$ )
```

Sortarea unei secvențe bitone – pseudocod

- *Premise:* Inițial, segmentul $A[i..i + d - 1]$ conține o secvență bitonă de lungime d ;

Algoritm 21: Algoritmul paralel de sortare a unei secvențe bitone

```

1: procedure SORTARE_SECVENTA_BITONA( $A, i, d, s$ )
2:   if  $d = 2$  then
3:      $(A[i], A[i + 1]) \leftarrow \text{COMPARA\_SI\_SCHIMBA}(A[i], A[i + 1], s)$ 
4:   else
5:     ► construirea secvențelor bitone  $S_1$  și  $S_2$  de lungime  $\frac{d}{2}$ 
6:     for all  $j: 0 \leq j < \frac{d}{2}$  parallel do
7:        $(A[i + j], A[i + j + \frac{d}{2}]) \leftarrow \text{COMPARA\_SI\_SCHIMBA}(A[i + j], A[i + j + \frac{d}{2}], s)$ 
8:     end for
9:     ► sortarea secvenței bitone  $S_1$ , de lungime  $\frac{d}{2}$ 
10:    SORTARE_SECVENTA_BITONA( $A, i, \frac{d}{2}, s$ )
11:    ► sortarea secvenței bitone  $S_2$ , de lungime  $\frac{d}{2}$ 
12:    SORTARE_SECVENTA_BITONA( $A, i + \frac{d}{2}, \frac{d}{2}, s$ )

```

Exemplu de sortare a unei secvențe bitone

Original

sequence	3	5	8	9	10	12	14	20	95	90	60	40	35	23	18	0
1st Split	3	5	8	9	10	12	14	0	95	90	60	40	35	23	18	20
2nd Split	3	5	8	0	10	12	14	9	35	23	18	20	95	90	60	40
3rd Split	3	0	8	5	10	9	14	12	18	20	35	23	60	40	95	90
4th Split	0	3	5	8	9	10	12	14	18	20	23	35	40	60	90	95

Copyright ©1994 Benjamin/Cummings Publishing Co.

Figura 33: Exemplu de sortare a unei secvențe bitone

Corectitudinea

Lema (2)

Dacă algoritmul lui Batchelor sortează orice secvență de chei de sortare binare, atunci sortează orice secvență de chei de sortare numere reale oarecare.

Demonstrație.

Folosim metoda reducerii la absurd.

Fie $f: \mathbb{R} \rightarrow \mathbb{R}$ o funcție monotonă. Astfel, $f(a_i) \leq f(a_j)$ dacă și numai dacă $a_i \leq a_j$. Evident, dacă algoritmul lui Batchelor transformă secvența $[a_1, a_2, \dots, a_n]$ în secvența $[b_1, b_2, \dots, b_n]$, atunci va transforma secvența $[f(a_1), f(a_2), \dots, f(a_n)]$ în secvența $[f(b_1), f(b_2), \dots, f(b_n)]$. Astfel, dacă în secvența $[b_1, b_2, \dots, b_n]$ există un indice i pentru care $b_i > b_{i+1}$, atunci în secvența $[f(b_1), f(b_2), \dots, f(b_n)]$ vom avea $f(b_i) > f(b_{i+1})$.

Fie acum f o funcție monotonă definită astfel:
$$f(b_j) = \begin{cases} 0 & , \text{dacă } b_j < b_i \\ 1 & , \text{dacă } b_j \geq b_i \end{cases}$$

În aceste condiții, secvența $[f(b_1), f(b_2), \dots, f(b_n)]$ va fi o secvență binară nesortată deoarece $f(b_i) = 1$ și $f(b_{i+1}) = 0$. Rezultă că algoritmul lui Batchelor eșuează în sortarea secvenței binare $[f(b_1), f(b_2), \dots, f(b_n)]$. Deci, dacă algoritmul lui Batchelor eșuează în sortarea unei secvențe de chei de sortare numere reale oarecare, atunci există o secvență binară care nu va fi sortată în urma aplicării algoritmului lui Batchelor. □

Corectitudinea – continuare

Teorema (4)

Algoritmul lui Batcher sortează cele n elemente ale secvenței memorate în tabloul $A[0..n-1]$, în ordinea crescătoare ($s = 0$), respectiv descrescătoare ($s = 1$) a cheilor de sortare.

Demonstrație.

Este suficient să demonstrăm corectitudinea procedurii `SORTARE_SECVENTA_BITONA` pentru cazul binar (Lema 2). Procedăm prin inducție după lungimea d a secvențelor procesate.

Dacă $d = 2$, evident procedura `SORTARE_SECVENTA_BITONA` transformă secvența inițială $S_{init} = A[i..i+d-1]$ într-o secvență sortată S_{fin} .

Vom demonstra că procedura `SORTARE_SECVENTA_BITONA` transformă o secvență binară S_{init} de tipul $0^r 1^t 0^v$ sau $1^r 0^t 1^v$ ($r + t + v = d$) într-o secvență S_{fin} sortată în ordinea indicată de valoarea lui s , $(\forall) d \geq 2$

Pasul paralel 6 transformă secvența S_{init} într-o secvență S_{temp} conform figurilor 34 și 35.

Se observă că în toate cazurile, secvența rezultată S_{temp} , este formată din două sub-secvențe bitone S_{temp}^1 și S_{temp}^2 , fiecare de lungime $\frac{d}{2}$. O secvență este de tipul S_{init} iar cealaltă secvență conține numai cifre 0 sau numai cifre 1. Dacă $s = 0$, cheia maximă din S_{temp}^1 este mai mică sau egală cu cheia minimă din S_{temp}^2 . Dacă $s = 1$, cheia minimă din S_{temp}^1 este mai mare sau egală cu cheia maximă din S_{temp}^2 .

Conform ipotezei de inducție, procedura `SORTARE_SECVENTA_BITONA` transformă secvențele S_{temp}^1 și S_{temp}^2 în două secvențe S_{fin}^1 și S_{fin}^2 , sortate crescător ($d = 0$) sau descrescător ($s = 1$). Dacă $s = 0$, cheia maximă din S_{fin}^1 este mai mică sau egală cu cheia minimă din S_{fin}^2 deci secvența $S_{fin}^1 S_{fin}^2$ este crescătoare. Dacă $s = 1$, cheia minimă din S_{fin}^1 este mai mare sau egală cu cheia maximă din S_{fin}^2 deci secvența $S_{fin}^1 S_{fin}^2$ este descrescătoare. □

Corectitudinea – continuare

s	$p + q(p, q \leq \frac{d}{2})$	Secvența inițială	Secvența rezultată
0	$\leq \frac{d}{2}$	$0 \frac{d}{2} - p 1 p 1 q 0 \frac{d}{2} - q$	$0 \frac{d}{2} 1 q 0 \frac{d}{2} - (p+q) 1 p$
0	$\leq \frac{d}{2}$	$0 \frac{d}{2} 0^k 1^{p+q} 0^m$	$0 \frac{d}{2} 0^k 1^{p+q} 0^m$
0	$\leq \frac{d}{2}$	$0^k 1^{p+q} 0^m 0 \frac{d}{2}$	$0 \frac{d}{2} 0^k 1^{p+q} 0^m$
0	$\leq \frac{d}{2}$	$1 p 0 \frac{d}{2} - p 0 \frac{d}{2} - q 1 q$	$0 \frac{d}{2} 1 p 0 \frac{d}{2} - (p+q) 1 q$
0	$\leq \frac{d}{2}$	$1 \frac{d}{2} 1 p 0 \frac{d}{2} - (p+q) 1 q$	$1 p 0 \frac{d}{2} - (p+q) 1 q 1 \frac{d}{2}$
0	$\leq \frac{d}{2}$	$1 p 0 \frac{d}{2} - (p+q) 1 q 1 \frac{d}{2}$	$1 p 0 \frac{d}{2} - (p+q) 1 q 1 \frac{d}{2}$
0	$> \frac{d}{2}$	$0 \frac{d}{2} - p 1 p 1 q 0 \frac{d}{2} - q$	$0 \frac{d}{2} - p 1 (p+q) - \frac{d}{2} 0 \frac{d}{2} - q 1 \frac{d}{2}$
0	$> \frac{d}{2}$	$1 p 0 \frac{d}{2} - p 0 \frac{d}{2} - q 1 q$	$0 \frac{d}{2} - q 1 (p+q) - \frac{d}{2} 0 \frac{d}{2} - p 1 \frac{d}{2}$
1	$\leq \frac{d}{2}$	$0 \frac{d}{2} - p 1 p 1 q 0 \frac{d}{2} - q$	$1 q 0 \frac{d}{2} - (p+q) 1 p 0 \frac{d}{2}$
1	$\leq \frac{d}{2}$	$0 \frac{d}{2} 0^k 1^{p+q} 0^m$	$0^k 1^{p+q} 0^m 0 \frac{d}{2}$
1	$\leq \frac{d}{2}$	$0^k 1^{p+q} 0^m 0 \frac{d}{2}$	$0^k 1^{p+q} 0^m 0 \frac{d}{2}$
1	$\leq \frac{d}{2}$	$1 p 0 \frac{d}{2} - p 0 \frac{d}{2} - q 1 q$	$1 p 0 \frac{d}{2} - (p+q) 1 q 0 \frac{d}{2}$
1	$\leq \frac{d}{2}$	$1 \frac{d}{2} 1 p 0 \frac{d}{2} - (p+q) 1 q$	$1 \frac{d}{2} 1 p 0 \frac{d}{2} - (p+q) 1 q$
1	$\leq \frac{d}{2}$	$1 p 0 \frac{d}{2} - (p+q) 1 q 1 \frac{d}{2}$	$1 \frac{d}{2} 1 p 0 \frac{d}{2} - (p+q) 1 q$
1	$> \frac{d}{2}$	$0 \frac{d}{2} - p 1 p 1 q 0 \frac{d}{2} - q$	$1 \frac{d}{2} 0 \frac{d}{2} - p 1 (p+q) - \frac{d}{2} 0 \frac{d}{2} - q$
1	$> \frac{d}{2}$	$1 p 0 \frac{d}{2} - p 0 \frac{d}{2} - q 1 q$	$1 \frac{d}{2} 0 \frac{d}{2} - q 1 (p+q) - \frac{d}{2} 0 \frac{d}{2} - p$

Figura 34: Corectitudinea

Corectitudinea – continuare

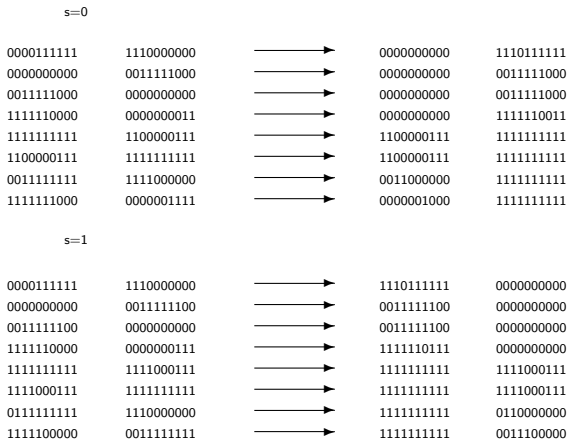
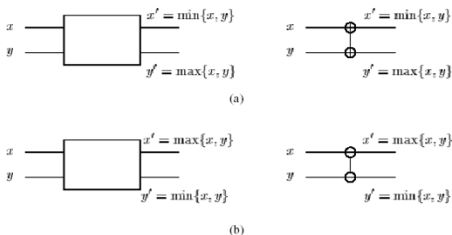


Figura 35: Corectitudinea

Implementarea algoritmului lui Batcher pe rețele de sortare

- ▶ Dacă se derecursivează algoritmul lui Batcher, se constată că sortarea unei secvențe de $n = 2^m$ elemente constă în m faze de sortare a unei secvențe bitone: $SSB_0, SSB_1, \dots, SSB_{m-1}$
- ▶ În faza SSB_k , $k \in \{0, 1, \dots, m-1\}$, se realizează sortarea secvențelor bitone $S_i^{2^d}$ formate din perechile de secvențe consecutive $S_i^d S_i'^d$ de lungime $d = 2^k$, S_i^d fiind sortată crescător și $S_i'^d$ descrescător.
- ▶ Secvențele $S_i^{2^d}$ cu numărul de ordine i par sunt sortate crescător. Cele cu i impar sunt sortate descrescător.



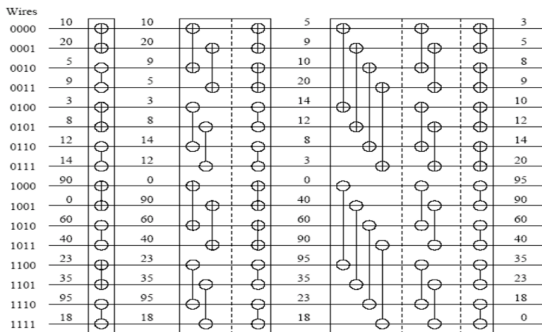
Copyright ©1994 Benjamin/Cummings Publishing Co.

Figura 36: Comparatori pentru sortarea a două elemente

Rețea de comparatori care transformă o secvență oarecare în una bitonă

Fazele $0, 1, \dots, m-2$

► Intrare: o secvență oarecare; ieșire: o secvență bitonă.

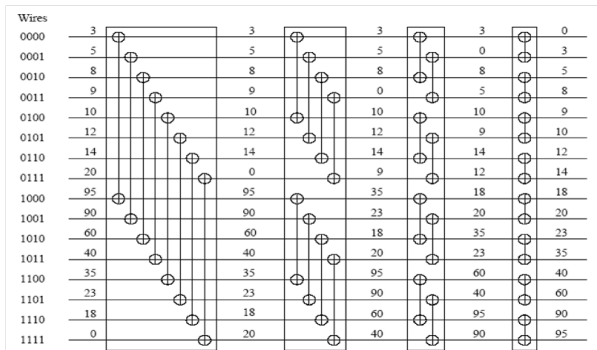


Copyright ©1994 Benjamin/Cummings Publishing Co.

Figura 37: Rețea de comparatori care transformă o secvență oarecare în una bitonă (R_1); $n = 16$

Rețea de sortare a unei secvențe bitone – Faza $m - 1$

- Intrare: o secvență bitonă; ieșire: o secvență sortată.



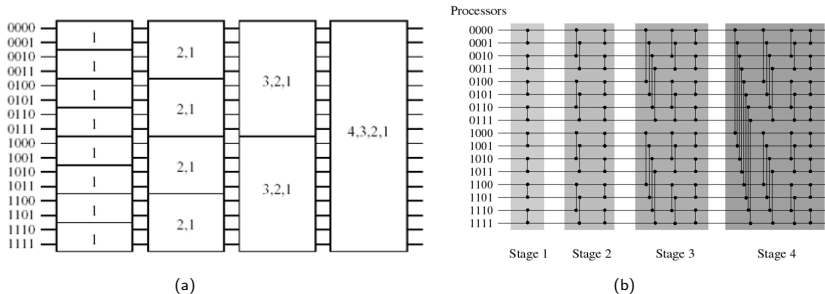
Copyright ©1994 Benjamin/Cummings Publishing Co.

Figura 38: Rețea de sortare a unei secvențe bitone (R_2); $n = 16$

Implementarea algoritmului lui Batcher pe hipercub

- ▶ Cubul binar multidimensional este o arhitectură ideală pentru implementarea algoritmului lui Batcher.
- ▶ Reamintim:
 - ▶ Dacă se derecursivează algoritmul lui Batcher, se constată că sortarea unei secvențe de $n = 2^m$ elemente constă în m faze de sortare a unei secvențe bitone: $SSB_0, SSB_1, \dots, SSB_{m-1}$
 - ▶ În faza SSB_k , $k \in \{0, 1, \dots, m-1\}$, se realizează sortarea secvențelor bitone $S_i^{2^d}$ formate din perechile de secvențe consecutive $S_i^d S_i'^d$ de lungime $d = 2^k$, S_i^d fiind sortată crescător și $S_i'^d$ descrescător.
 - ▶ Secvențele $S_i^{2^d}$ cu numărul de ordine i par sunt sortate crescător. Cele cu i impar sunt sortate descrescător.
- ▶ Execuția fazei SSB_k pe hipercub necesită utilizarea succesivă a dimensiunilor $D_k, D_{k-1}, \dots, D_1, D_0$.
- ▶ Planificarea utilizării dimensiunilor pentru o sortare completă poate fi reprezentată astfel:
 - ▶ $SSB_0 : D_0$
 - ▶ $SSB_1 : D_1, D_0$
 - ▶ $SSB_2 : D_2, D_1, D_0$
 - ▶ ...
 - ▶ $SSB_{m-1} : D_{m-1}, D_{m-2}, \dots, D_0$

Planificarea dimensiunilor = Fazele sortării prin rețele de sortare Batcher

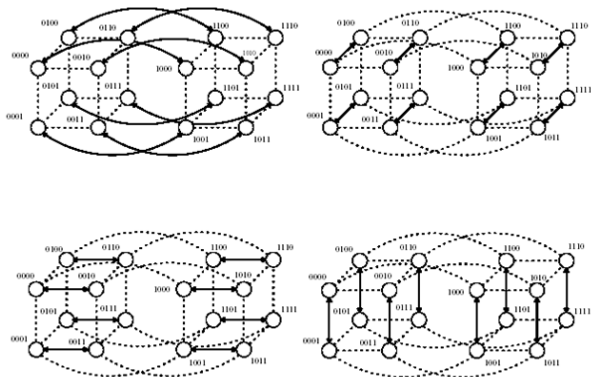


Copyright ©1994 Benjamin/Cummings Publishing Co.

Figura 39: (a) Planificarea dimensiunilor (b) Fazele sortării prin rețele de sortare Batcher

Comunicarea pe hipercub în timpul ultimei faze a algoritmului lui Batcher

Fiecare linie continuă reprezintă o operație de interschimbare.



Copyright ©1994 Benjamin/Cummings Publishing Co.

Figura 40: Comunicarea în timpul ultimei faze a algoritmului lui Batcher

Complexitatea implementării pe o mașină CREW-PRAM

Teorema (4)

Complexitatea timp a algoritmului lui Batcher de sortare paralelă, implementat pe mașină CREW-PRAM cu $O(n)$ unități de procesare este $O(\log^2 n)$. Eficiența algoritmului este $O(\frac{1}{\log n})$.

Demonstrație.

Timpul paralel pentru sortarea unei secvențe bitone de lungime $d < n$ este $O(\log d)$. După $\log n$ faze algoritmul se termină. Eficiența este $\frac{O(n \log n)}{n O(\log^2 n)} = O(\frac{1}{\log n})$. □

Complexitatea implementării pe rețele de sortare

Teorema (5)

Complexitatea timp a algoritmului lui Batcher de sortare paralelă, implementat pe o rețea de sortare cu $O(n)$ intrări și ieșiri, este $O(\log^2 n)$.

Demonstrație.

Rețeaua (R_1, R_2) implementează algoritmul lui Batcher. Numărul de faze este $\log n$. Fazele sunt compuse din $\log d < \log n$ pași □

Complexitatea implementării pe hipercub

Teorema (6)

Complexitatea timp a algoritmului lui Batcher de sortare paralelă, implementat pe un hipercub cu $O(n)$ unități de procesare este $O(\log^2 n)$.

Demonstrație.

Sortarea Batcher pe hipercub este congruentă cu sortarea Batcher pe rețele de sortare



Comentarii

- ▶ Algoritmul lui Batcher de sortare paralelă este din clasa timp $O(\log^2 n)$. Numărul de unități de procesare este din clasa $O(n)$. Comparativ cu algoritmiile Muller-Preparata și Impar-Par are un cost mai bun.
- ▶ Totuși, costul nu este optimal: (Numărul de unități de procesare) $\times$ (timpul paralel) $= n \log^2 n = O(n \log^2 n)$. Timpul pentru cel mai rapid algoritm secvențial este $O(n \log n)$.

Sortare rapidă pe hipercub - descriere

- ▶ Să ne amintim că un hipercub cu m dimensiuni este format din două hipercuburi cu $m - 1$ dimensiuni.
- ▶ Numărul unităților de procesare, p , este mai mic decât numărul elementelor secvenței de sortat, n .
- ▶ Ideea este de a partiționa secvența de sortat pe subcuburi și apoi de a repeta repeta recursiv această operație.
- ▶ Selectarea pivotului este problema cheie.

Sortare rapidă pe hipercub - pseudocod

► *Notatii:*

- $A[0..n-1]$ este un tablou de dimensiune $n = 2^m$.

► *Premise:*

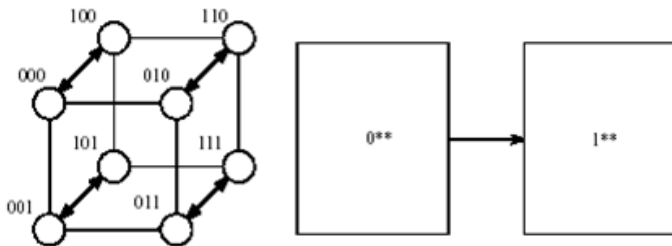
- Datele de intrare sunt partiționate între unitățile de procesare.
- Fiecare unitate de procesare (nod al hipercubului) memorează o parte A_i din secvența de sortat, memorată inițial în tabloul $A[0..n-1]$.

Algoritm 22: Algoritmul de sortare rapidă pe hipercub

```

1: procedure SORTARE_RAPIDA_PE_HIPERCUB( $A_i, m, p_i, pivot$ )
2:    $i \leftarrow$  eticheta unitatii de procesare  $p_i$ 
3:   for  $k \leftarrow m-1 \rightarrow 0$  do
4:      $x \leftarrow pivot$ 
5:     partitioneaza  $A_i$  in  $A_{i_1}, A_{i_2}$  astfel incat  $A_{i_1} \leq x \leq A_{i_2}$ 
6:     if  $bit_k(i) = 0$  then
7:       trimite  $A_{i_2}$  unitatii vecina pe dimensiunea  $k$ 
8:        $B_{i_1} \leftarrow$  subsecvența primita de la vecinul pe dimensiunea  $k$ 
9:        $A_i \leftarrow A_{i_1} \cup B_{i_1}$ 
10:    else
11:      trimite  $A_{i_1}$  unitatii vecina pe dimensiunea  $k$ 
12:       $B_{i_2} \leftarrow$  subsecvența primita de la vecinul pe dimensiunea  $k$ 
13:       $A_i \leftarrow A_{i_2} \cup B_{i_2}$ 
14:   aplica lui  $A_i$  sortarea rapida secventiala
```

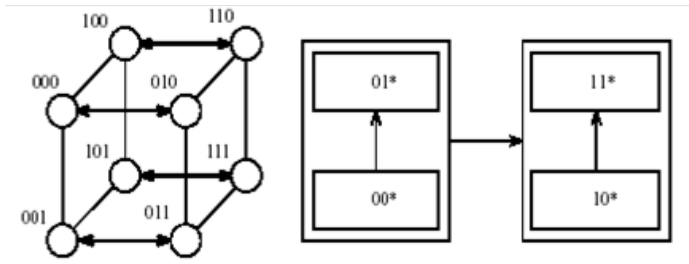
Exemplu de execuție a algoritmului de sortare rapidă pe hipercub ($m=3$)



Copyright ©1994 Benjamin/Cummings Publishing Co.

Figura 41: Partiționarea secvenței de sortat în două blocuri. Se utilizează dimensiunea a treia ($m-1$).

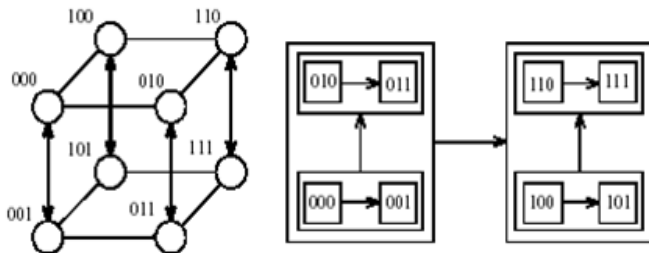
Exemplu de execuție a algoritmului de sortare rapidă pe hipercub - continuare



Copyright ©1994 Benjamin/Cummings Publishing Co.

Figura 42: Partiționarea fiecărui bloc în două sub-blocuri. Se utilizează dimensiunea a doua ($m - 2 = 1$).

Exemplu de execuție a algoritmului de sortare rapidă pe hipercub - continuare



Copyright ©1994 Benjamin/Cummings Publishing Co.

Figura 43: Partiționarea sub-blocurilor. Se utilizează prima dimensiune ($m - 3 = 0$).

Complexitatea

Teorema (7)

Dacă pivotul este ales astfel încât să partiționeze secvența în două subsecvențe de dimensiuni aproximativ egale, atunci $T_p = O(\frac{n}{p} \log \frac{n}{p}) + O(\frac{n}{p} \log p) + O(\log^2 p)$.

Comentarii

- Selectarea unui pivot care să partiționeze secvența în două subsecvențe de dimensiuni aproximativ egale este dificilă.

- Capitolul sortare are la bază cartea

V. Kumar, A. Grama, A. Gupta & G Karypis, Introduction to Parallel Computing: Design and Analysis of Algorithms, Addison Wesley, 2003

și ediția mai veche

V. Kumar, A. Grama, A. Gupta & G Karypis, Introduction to Parallel Computing: Design and Analysis of Algorithms, Benjamin-Cummings, 1994

Cuprins

1. Introducere
2. Modele de comunicare
3. Topologii de comunicare
4. Modele de calcul paralel/distribuit
5. Parametri de performanță a unui algoritm paralel/distribuit
6. Comunicarea în sistemele de calcul paralel/distribuit
7. Algoritmi fundamentali
8. Sortare paralelă
- 9. Calcul matriceal**
10. Sisteme de ecuații liniare
11. Alegerea liderului
12. Excluderea mutuală în sisteme *shared-memory*

Calcul matriceal

- ▶ Calculul matricial se pretează extrem de bine la procesare paralelă.
- ▶ Natura regulată a matricelor constituie un atu de neegalat pentru abordări specifice calculului paralel.
- ▶ Vom studia în această lecție problemele transpusei unei matrice și produsului de matrice.

Transpusa unei matrice – formularea problemei

- ▶ Dată fiind matricea pătratică $A_{n \times n} = (a_{i,j})_{i,j=0,1,\dots,n-1}$, se cere să se calculeze matricea $A_{n \times n}^T = (a_{i,j}^T)_{i,j=0,1,\dots,n-1}$, pentru care $a_{i,j}^T = a_{j,i}$, $(\forall) i,j = 0,1,\dots,n-1$.
- ▶ Fără calcule.
- ▶ Doar mișcări de elemente.
- ▶ Timpul secvențial $T_s(n) = O(n^2)$.

Algoritmul secvențial standard – pseudocod

► *Notatii:*

- $A[0..n-1, 0..n-1]$ este un tablou bidimensional, de dimensiune $n \times n$.

► *Premise:*

- Datele de intrare sunt memorate în tabloul A .
- Datele finale vor fi memorate în tabloul A .

Algoritm 23: Algoritmul secvențial

```
1: procedure TRANSPUNERE_MATRICE( $A, n$ )
2:   for  $i \leftarrow 0 \rightarrow n-1$  do
3:     for  $j \leftarrow i+1 \rightarrow n-1$  do
4:       INTERSCHIMBA( $A[i,j], A[j,i]$ )
```

Algoritm 24: Interschimbare

```
1: procedure INTERSCHIMBA( $A[i,j], A[j,i]$ )
2:    $temp \leftarrow A[i,j]$ 
3:    $A[i,j] \leftarrow A[j,i]$ 
4:    $A[j,i] \leftarrow temp$ 
```

Exemplu de execuție a algoritmului de transpunere a unei matrice pe o plasă de unități de procesare

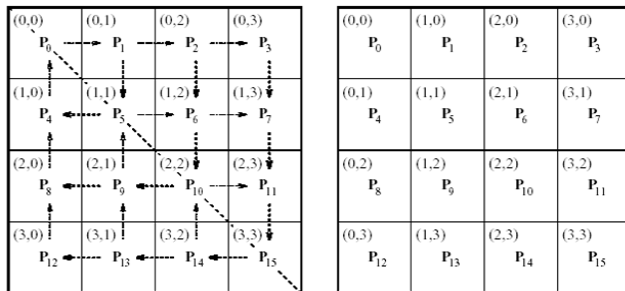


Figura 44: Exemplu de execuție a algoritmului de transpunere a unei matrice 4x4 pe o plasă de unități de procesare

Algoritmul recursiv – pseudocod

► Notatii:

- $A[0..n-1, 0..n-1]$ este un tablou bidimensional, de dimensiune $n \times n$.

► Premise:

- $n = 2^m$.
- Datele de intrare sunt memorate în tabloul A .
- Matricea A este divizată în patru blocuri: *stânga-sus*, *dreapta-sus*, *stânga-jos*, *dreapta-jos*.
- Datele finale vor fi memorate în tabloul A .

Algoritm 25: Algoritmul recursiv

1: **procedure** TRANSPUNERE_MATRICE_RECURSIV($A[0 \dots n-1, 0 \dots n-1]$)

 ► Interschimbarea blocurilor *stânga-jos* cu *dreapta-sus*

2: **for** $i \leftarrow \frac{n}{2} \rightarrow n-1$ **do**

3: **for** $j \leftarrow 0 \rightarrow \frac{n}{2}-1$ **do**

4: INTERSCHIMBA($A[i, j]$, $A[i - \frac{n}{2}, j + \frac{n}{2}]$)

5: TRANSPUNERE_MATRICE_RECURSIV($A[0 \dots \frac{n}{2}-1, 0 \dots \frac{n}{2}-1]$)

6: TRANSPUNERE_MATRICE_RECURSIV($A[0 \dots \frac{n}{2}-1, \frac{n}{2} \dots n-1]$)

7: TRANSPUNERE_MATRICE_RECURSIV($A[\frac{n}{2} \dots n-1, 0 \dots \frac{n}{2}-1]$)

8: TRANSPUNERE_MATRICE_RECURSIV($A[\frac{n}{2} \dots n-1, \frac{n}{2} \dots n-1]$)

Exemplu de execuție a algoritmului recursiv de transpunere a unei matrice

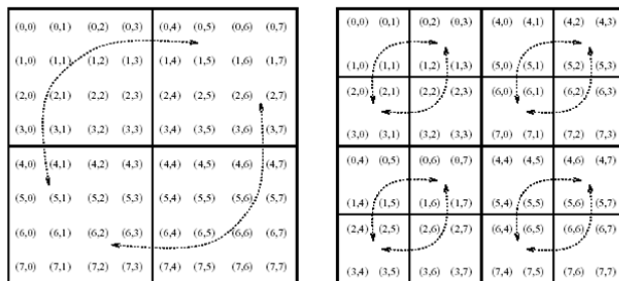


Figura 45: Exemplu de execuție a algoritmului recursiv de transpunere a unei matrice 8x8

Exemplu de execuție a algoritmului recursiv de transpunere a unei matrice – continuare

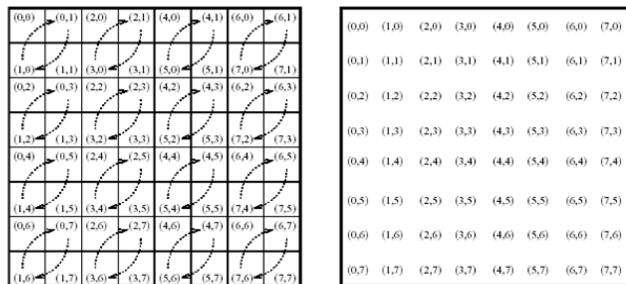


Figura 46: Exemplu de execuție a algoritmului recursiv de transpunere a unei matrice 8x8

Implementare pe hipercub

1. Hipercubul este divizat în 4 subcuburi cu $\frac{P}{4}$ procesoare.
2. Sferturile de matrice (quadranții) sunt mapate recursiv pe cele 4 subcuburi astfel:
 - ▶ cubul 00* conține sfertul *stânga-sus* al matricei A ;
 - ▶ cubul 01* conține sfertul *dreapta-sus* sfert al matricei A ;
 - ▶ cubul 10* conține sfertul *stânga-jos* al matricei A ;
 - ▶ cubul 11* conține sfertul *dreapta-jos* al matricei A .
3. Se interschimbă blocurile *stânga-jos* cu *dreapta-sus*.
4. Se repetă pașii 1-3 în fiecare subcub.

Algoritmul recursiv implementat pe hipercub – pseudocod

► *Notatii:*

- H este un hipercub cu m dimensiuni; i este indicele unității de procesare care execută algoritmul de transpunere a unei matrice A .
- M este blocul care urmează a fi transmis de o unitate de procesare către partener.
- xor_b = sau exclusiv pe biți; and_b = și pe biți; and = și logic; or = sau logic

► *Premise:*

- Inițial, fiecare unitate de procesare p_i deține un bloc B_i din matricea A , conform cu regula de mapare, care urmează a fi transmis unei unități de procesare plasate într-un nod al hipercubului H .

Algoritmul recursiv implementat pe hipercub – pseudocod

Algoritm 26: Algoritmul recursiv implementat pe hipercub

```

1: procedure TRANSPUNERE_MATRICE_PE_HIPERCUB( $H, m, i, B_i$ )
2:    $M \leftarrow B_i$ 
3:   for  $k \leftarrow m - 1 \rightarrow 0$ , step 2 do
4:      $partener \leftarrow i \text{ xor}_b 2^k$ 
5:     if  $((i \text{ and}_b 2^k) \neq 0 \text{ and } (i \text{ and}_b 2^{k-1}) = 0) \text{ or } ((i \text{ and}_b 2^k) = 0 \text{ and } (i \text{ and}_b 2^{k-1}) \neq 0)$  then
6:       trimite M catre partener
7:     else
8:       primeste M catre partener
9:      $partener \leftarrow i \text{ xor}_b 2^{k-1}$ 
10:    if  $((i \text{ and}_b 2^k) = 0 \text{ and } (i \text{ and}_b 2^{k-1}) = 0) \text{ or } ((i \text{ and}_b 2^k) \neq 0 \text{ and } (i \text{ and}_b 2^{k-1}) \neq 0)$  then
11:      trimite M catre partener
12:    else
13:      primeste M catre partener
14:       $B_i \leftarrow M$ 
15:  TRANSPUNERE_MATRICE( $B_i, \frac{n^2}{p}$ )

```

▷ Algoritmul 23

Exemplu de execuție a algoritmului de transpunere a unei matrice pe hipercub

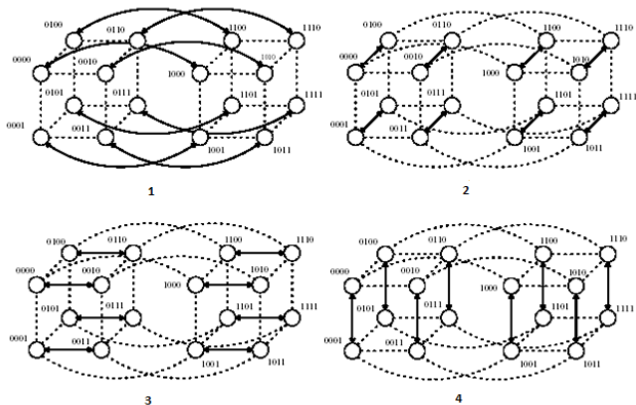


Figura 47: Exemplu de execuție a algoritmului de transpunere pe hipercub a unei matrice 4x4

Complexitatea implementării pe hipercub algoritmului recursiv de transpunere a unei matrice

Teorema (1)

Complexitatea timp a algoritmului recursiv de transpunere a unei matrice $A_{n \times n}$, implementat pe un hipercub cu p unități de procesare este $O(\frac{n^2}{p} \log p)$. Eficiența algoritmului este $O(\frac{1}{\log n})$.

Demonstrație.

Divizarea recursivă se oprește când dimensiunea blocului este $\frac{n^2}{p}$. Numărul de pași de interschimbare de blocuri este $\log_4 p = \frac{\log_2 p}{2}$; fiecare pas de interschimbare se realizează pe două dintre dimensiunile hipercubului (două muchii). Timpul de transfer al unui bloc de dimensiune $\frac{n^2}{p}$ este $O(\frac{n^2}{p})$. Transpunerea locală necesită $O(\frac{n^2}{p})$ timp.

Rezultă că timpul total este $O(\frac{n^2}{p} \log p)$. Costul este $O(n^2 \log p)$ – nu este optimal. □

Înmulțirea de matrice pătratice – formularea problemei

- Date fiind matricele pătratice $A_{n \times n} = (a_{i,j})_{i,j=0,1,\dots,n-1}$ și $B_{n \times n} = (b_{i,j})_{i,j=0,1,\dots,n-1}$, se cere să se calculeze matricea pătratică $C_{n \times n} = (c_{i,j})_{i,j=0,1,\dots,n-1}$, conform cu formula

$$c_{i,j} = \sum_{k=0}^{n-1} a_{i,k} \times b_{k,j}, \quad (\forall) i,j = 0,1,\dots,n-1.$$

- Timpul secvențial $T_s(n) = O(n^3)$.

Algoritmul secvențial standard – pseudocod

► *Notatii:*

- $A[0..n-1, 0..n-1]$, $B[0..n-1, 0..n-1]$ și $C[0..n-1, 0..n-1]$ sunt tablouri bidimensionale, de dimensiune $n \times n$.

► *Premise:*

- Datele de intrare sunt memorate în tablourile A și B ;
 $A[i, j] = a_{ij}$, $B[i, j] = b_{ij}$, $i, j = 0, 1, \dots, n-1$.
► Datele finale vor fi memorate în tabloul C .

Algoritm 27: Algoritmul secvențial standard

```
1: procedure INMULTIRE_MATRICE( $A, B, n$ )
2:   for  $i \leftarrow 0 \rightarrow n-1$  do
3:     for  $j \leftarrow 0 \rightarrow n-1$  do
4:        $C[i, j] \leftarrow 0$ 
5:       for  $k \leftarrow 0 \rightarrow n-1$  do
6:          $C[i, j] \leftarrow C[i, j] + A[i, k] \times B[k, j]$ 
7:   return  $C$ 
```

Algoritmul înmulțirii de blocuri – pseudocod

► *Notatii:*

- $A[0..n-1, 0..n-1]$, $B[0..n-1, 0..n-1]$ și $C[0..n-1, 0..n-1]$ sunt tablouri bidimensionale, de dimensiune $n \times n$.
- Matricele A și B sunt divizate în blocuri $A_{i,k}$ și respectiv $B_{k,j}$, de dimensiuni $\frac{n}{q} \times \frac{n}{q}$.
- $\text{INMULTIRE_MATRICE}(A_{i,k}, B_{k,j}, \frac{n}{q})$ semnifică înmulțirea blocului $A_{i,k}$ cu $B_{k,j}$ și returnarea rezultatului $A_{i,k} \times B_{k,j}$.

► *Premise:*

- Datele de intrare sunt memorate în tablourile A și B .
- Datele finale vor fi memorate în tabloul C .

Algoritm 28: Algoritmul înmulțirii de blocuri

```

1: procedure INMULTIRE_MATRICE_BLOCURI( $A, B, n, q$ )
2:   for  $i \leftarrow 0 \rightarrow q-1$  do
3:     for  $j \leftarrow 0 \rightarrow q-1$  do
4:        $C[i,j] \leftarrow 0$ 
5:       for  $k \leftarrow 0 \rightarrow q-1$  do
6:          $C[i,j] \leftarrow C[i,j] + \text{INMULTIRE\_MATRICE}(A_{i,k}, B_{k,j}, \frac{n}{q})$ 
7:   return  $C$ 
```

► Algoritmul 27

Implementarea standard

- ▶ Arhitectura naturală este cea cu topologie de tip plasă, cu $p = q^2$ unități de procesare.
- ▶ Fiecare unitate de procesare $p_{i,j}$ dispune în memoria locală de blocul $A_{i,j}$ și un bloc $B_{i,j}$, și calculează $C_{i,j}$.
- ▶ Pentru a calcula $C_{i,j}$, unitatea de procesare $p_{i,j}$ are nevoie de $A_{i,k}$ și $B_{k,j}$, $k = 0, 1, \dots, q - 1$.
- ▶ Pentru fiecare k , blocurile $A_{i,k}$ și $B_{k,j}$ sunt obținute printr-o difuzie *toți-la-toți* pe linii și apoi pe coloane.

Implementarea lui Cannon

- ▶ Similar cu implementarea standard, dar cu consum mai mic de memorie.
- ▶ Fiecare bloc este procesat la momente diferite, în locuri diferite. Pentru aceasta blocurile sunt deplasate ciclic.
- ▶ Sunt executați $\sqrt{p} - 1$ pași de înmulțiri și adunări locale de blocuri, urmate de deplasări ciclice la stânga (în A) și în sus (în B).
- ▶ La final este executată o înmulțire și o adunare locală de blocuri.

Algoritmul lui Cannon – pseudocod

► *Notății:*

- P este o plasă formată din $q \times q = \sqrt{p} \times \sqrt{p} = p$ unități de procesare.
 - $p_{i,j}$ este unitatea de procesare care execută algoritmul de înmulțire de blocuri.
 - M_1 este blocul care urmează a fi transmis de $p_{i,j}$ către $p_{i,j \oplus 1}$.
 - M_2 este blocul care urmează a fi transmis de $p_{i,j}$ către $p_{i \oplus 1, j}$.
 - M_3 este blocul calculat de $p_{i,j}$.
 - $\oplus$ și $\ominus$ semnifică adunarea și scăderea modulo q .
-
- *Premise:* Inițial, fiecare $p_{i,j}$ deține în memoria locală blocurile $A_{i,j}$ și $B_{i,j}$.
 - *Rezultate:* Fiecare unitate de procesare $p_{i,j}$ calculează blocul $C_{i,j}$.

Algoritmul lui Cannon – pseudocod

Algoritm 29: Algoritmul lui Cannon

```

1: procedure INMULTIRE_MATRICE_CANNON( $A_{i,j}$ ,  $B_{i,j}$ ,  $n$ ,  $q$ ,  $p_{i,j}$ )
2:    $M_1 \leftarrow A_{i,j}$ 
3:    $M_2 \leftarrow B_{i,j}$ 
4:   ALINIERE_INITIALA( $M_1$ ,  $M_2$ ,  $n$ ,  $q$ ,  $p_{i,j}$ )
5:    $M_3 \leftarrow 0$ 
   ▷  $q - 1$  pasi de inmultiri si adunari de blocuri
6:   for  $k \leftarrow 0 \rightarrow q - 2$  do
7:      $M_3 \leftarrow M_3 + \text{INMULTIRE\_MATRICE}(M_1, M_2, q)$ 
8:     trimite  $M_1$  catre  $p_{i,j \oplus 1}$ 
9:     primește  $M_1$  de la  $p_{i,j \oplus 1}$ 
10:    trimite  $M_2$  catre  $p_{i \oplus 1,j}$ 
11:    primește  $M_2$  de la  $p_{i \oplus 1,j}$ 
12:    $C_{i,j} \leftarrow M_3 + \text{INMULTIRE\_MATRICE}(M_1, M_2, q)$ 
13:   return  $C_{i,j}$ 

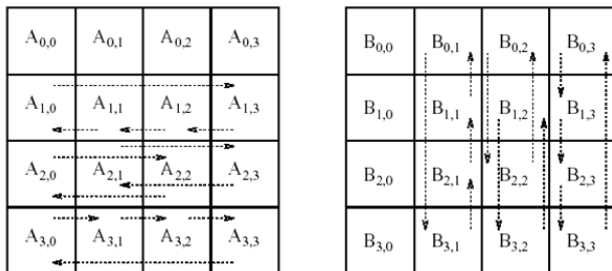
```

Algoritmul lui Cannon – pseudocod

Algoritm 30: Algoritmul pentru alinierea inițială

```
1: procedure ALINIERE__INITIALA( $M_1, M_2, n, q, p_{i,j}$ )
2:   for  $k \leftarrow 1 \rightarrow i$  do
3:     trimite  $M_1$  catre  $p_{i,j \ominus 1}$ 
4:     primește  $M_1$  de la  $p_{i,j \oplus 1}$ 
5:   for  $k \leftarrow 1 \rightarrow j$  do
6:     trimite  $M_2$  catre  $p_{i \ominus 1,j}$ 
7:     primește  $M_2$  de la  $p_{i \oplus 1,j}$ 
```

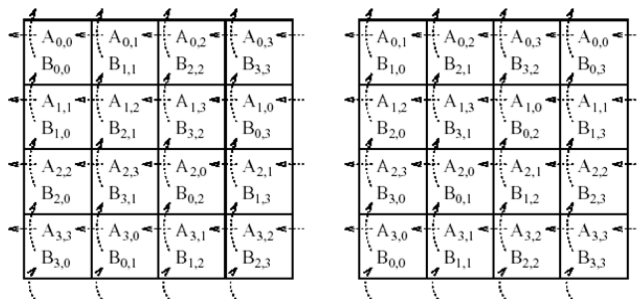
Exemplu de execuție a algoritmului lui Cannon



Alinierea inițială

Figura 48: Exemplu de execuție a algoritmului lui Cannon

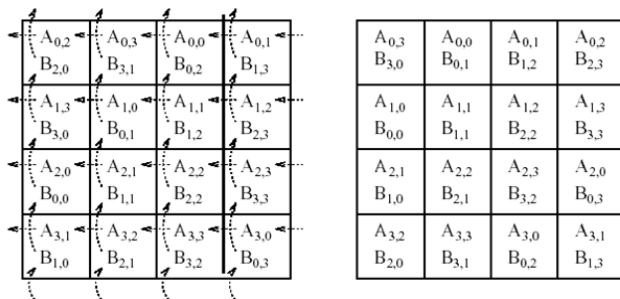
Exemplu de execuție a algoritmului lui Cannon



A și B după alinierea inițială Pozițiile blocurilor după prima deplasare

Figura 49: Exemplu de execuție a algoritmului lui Cannon

Exemplu de execuție a algoritmului lui Cannon



Pozițiile blocurilor după a II-a deplasare Pozițiile blocurilor după a III-a deplasare

Figura 50: Exemplu de execuție a algoritmului lui Cannon

Complexitatea algoritmului lui Cannon

Teorema (2)

Complexitatea timp a algoritmului lui Cannon este $O(\frac{n^3}{p})$. Eficiența algoritmului este $O(1)$.

Demonstrație.

Numărul de transmisii și primiri de blocuri M_1 efectuate de $p_{i,j}$ este $i + q - 1 = i + \sqrt{p} - 1$. Numărul de transmisii și primiri de blocuri M_2 efectuate de $p_{i,j}$ este tot $j + q - 1 = j + \sqrt{p} - 1$. Se observă că $p_{q-1,q-1}$ efectuează cele mai multe transmisii și primiri de blocuri, $q - 1 + q - 1 = 2(q - 1) = 2(\sqrt{p} - 1)$. Rezultă pentru deplasările de blocuri o complexitate timp de $O(\frac{n^2}{p} \sqrt{p})$. Numărul înmulțirilor și adunărilor efectuate de o unitate de procesare este $O((\frac{n}{\sqrt{p}})^3 \sqrt{p}) = O(\frac{n^3}{p})$; $O((\frac{n}{\sqrt{p}})^3)$ pentru fiecare înmulțire și adunare de blocuri. Costul este $O(p)O(\frac{n^3}{p}) = O(n^3)$. □

Limitări ale algoritmului lui Cannon

- Dimensiunea matricelor n trebuie să fie divizibilă cu $\sqrt{p}$ (p este numărul de unități de procesare disponibile).
- Adaptarea algoritmului pentru scenariile în care matricele nu sunt pătratice este dificilă.

Observație

Considerăm cazul $C_{n,m} = A_{n,k} \times B_{k,m}$. Varianta *naivă* de rezolvare a acestor scenarii constă în identificarea unei valori $n' \geq \max(n, k, m)$, astfel încât $n' = \sqrt{p} \times \sqrt{p}$. În continuare, se completează elementele lipsă cu valori nule, apoi se aplică algoritmul lui Cannon.

Algoritmul SUMMA – cazul naiv

► *Notatii:*

- Considerăm cazul $C_{n,m} = A_{n,l} \times B_{l,m}$.
- Unitățile de procesare $p_{i,j}$ sunt dispuse pe o plasă de dimensiune $p = n \times m$.

► *Premise:* Inițial, fiecare $p_{i,j}$ deține în memoria locală:

- $[l/m]$ elemente de pe linia i din A (minim un element pentru primele l unități de procesare de pe fiecare linie, în cazul în care $[l/m] = 0$).
- $[l/n]$ elemente de pe coloana j din B (minim un element pentru primele k unități de procesare de pe fiecare coloană, în cazul în care $[l/n] = 0$).

► *Rezultate:* Fiecare unitate de procesare $p_{i,j}$ calculează elementul $C[i,j]$.

Algoritmul SUMMA – pseudocod

Algoritm 31: Algoritmul SUMMA – p_{ij} calculează $C[i,j]$

```
1: procedure INMULTIRE_MATRICE_SUMMA( $A, B, n, l, m, p_{ij}$ )
2:    $C[i,j] \leftarrow 0$ 
3:   for  $k \leftarrow 0 \rightarrow l - 1$  do
4:     transmite  $A[i,k]$  către toate unitățile de procesare de pe linia  $i$ 
5:     transmite  $B[k,j]$  către toate unitățile de procesare de pe coloana  $j$ 
6:      $C[i,j] \leftarrow C[i,j] + A[i,k] \times B[k,j]$ 
7:   return  $C[i,j]$ 
```

Algoritmul SUMMA – exemplificare

Exemplu:

- ▶ $C_{4,4} = A_{4,2} \times B_{2,4}$ ($n = 4, l = 2, m = 4$)
- ▶ Se utilizează 16 unități de procesare, dispuse pe topologie de tip plasă periodică.
- ▶ Liniile punctate marchează celulele lipsă din matricea A , respectiv B .

Distribuția inițială a valorilor

A (4x2)	0	1	2	3
0	P_0 A[0,0]	P_1 A[0,1]	P_2	P_3
1	P_4 A[1,0]	P_5 A[1,1]	P_6	P_7
2	P_8 A[2,0]	P_9 A[2,1]	P_{10}	P_{11}
3	P_{12} A[3,0]	P_{13} A[3,1]	P_{14}	P_{15}

B (2x4)	0	1	2	3
0	P_0 B[0,0]	P_1 B[0,1]	P_2 B[0,2]	P_3 B[0,3]
1	P_4 B[1,0]	P_5 B[1,1]	P_6 B[1,2]	P_7 B[1,3]
2	P_8	P_9	P_{10}	P_{11}
3	P_{12}	P_{13}	P_{14}	P_{15}

Figura 51: Distribuția inițială a matricelor A și B pe plasa de procesoare

Algoritmul SUMMA – exemplificare

Exemplu:

- Liniiile punctate marchează celulele lipsă din matricea A , respectiv B .
- Elementele marcate prin **albastru** reprezintă valorile distribuite în difuziile pe linii/coloane în matricele A/B .

Iterația $k=0$

A (4x2)		0	1	2	3
0	P0	A[0,0]	P1	P2	P3
	A[0,0]	A[0,1]			
1	P4	A[1,0]	P5	P6	P7
	A[1,0]	A[1,1]			
2	P8	A[2,0]	P9	P10	P11
	A[2,0]	A[2,1]			
3	P12	A[3,0]	P13	P14	P15
	A[3,0]	A[3,1]			

B (2x4)		0	1	2	3
0	P0	B[0,0]	P1	P2	P3
	B[0,0]	B[0,1]	B[0,2]	B[0,3]	
1	P4	B[1,0]	P5	P6	P7
	B[1,0]	B[1,1]	B[1,2]	B[1,3]	
2	P8		P9	P10	P11
3	P12		P13	P14	P15

Figura 52: Distribuția valorilor în pasul $k = 0$

Algoritmul SUMMA – exemplificare

Exemplu:

- Liniiile punctate marchează celulele lipsă din matricea A , respectiv B .
- Elementele marcate prin **albastru** reprezintă valorile distribuite în difuziile pe linii/coloane în matricele A/B .

Iterația $k=1$

A (4x2)		0	1	2	3
0	P0	A[0,0]	A[0,1]	P2	P3
		A[0,1]	A[0,1]	A[0,1]	A[0,1]
1	P4	A[1,0]	A[1,1]	P6	P7
		A[1,1]	A[1,1]	A[1,1]	A[1,1]
2	P8	A[2,0]	A[2,1]	P10	P11
		A[2,1]	A[2,1]	A[2,1]	A[2,1]
3	P12	A[3,0]	A[3,1]	P14	P15
		A[3,1]	A[3,1]	A[3,1]	A[3,1]

B (2x4)		0	1	2	3
0	P0	B[0,0]	B[0,1]	B[0,2]	B[0,3]
		B[1,0]	B[1,1]	B[1,2]	B[1,3]
1	P4	B[1,0]	B[1,1]	B[1,2]	B[1,3]
		B[1,0]	B[1,1]	B[1,2]	B[1,3]
2	P8				
		B[1,0]	B[1,1]	B[1,2]	B[1,3]
3	P12				
		B[1,0]	B[1,1]	B[1,2]	B[1,3]

Figura 53: Distribuția valorilor în pasul $k = 1$

Algoritmul SUMMA

Exercițiu

Evaluati timpul de calcul paralel, eficiența și costul algoritmului SUMMA pentru cazul naiv (Algoritmul 31).

Comentarii bibliografice

- ▶ Capitolul *Calcul matricial* are la bază cartea
V. Kumar, A. Grama, A. Gupta & G Karypis, Introduction to Parallel Computing: Design and Analysis of Algorithms, Addison Wesley, 2003
și ediția mai veche
V. Kumar, A. Grama, A. Gupta & G Karypis, Introduction to Parallel Computing: Design and Analysis of Algorithms, Benjamin-Cummings, 1994
- ▶ Discuțiile premergătoare și Algoritmul 31 au la bază articolul
van de Geijn R. A. & Watts J., SUMMA: scalable universal matrix multiplication algorithm, Concurrency: Practice and Experience, Volume 9, Issue 4, Apr 1997, Pages 241-317

Cuprins

1. Introducere
2. Modele de comunicare
3. Topologii de comunicare
4. Modele de calcul paralel/distribuit
5. Parametri de performanță a unui algoritm paralel/distribuit
6. Comunicarea în sistemele de calcul paralel/distribuit
7. Algoritmi fundamentali
8. Sortare paralelă
9. Calcul matriceal
- 10. Sisteme de ecuații liniare**
11. Alegerea liderului
12. Excluderea mutuală în sisteme *shared-memory*

Sisteme de ecuații liniare

- ▶ Sistemele de ecuații liniare pot fi rezolvate prin tehnici de calcul paralel datorită multiplelor operații independente una de alta.
- ▶ Vom studia în această lecție metoda lui Gauss de rezolvare a sistemelor de ecuații liniare.

Rezolvarea sistemelor de ecuații liniare prin metoda lui Gauss

- Se consideră un sistem format din n ecuații liniare cu n necunoscute:

$$a_{0,0}x_0 + a_{0,1}x_1 + \cdots + a_{0,n-1}x_{n-1} = b_0$$

$$a_{1,0}x_0 + a_{1,1}x_1 + \cdots + a_{1,n-1}x_{n-1} = b_1$$

.....

$$a_{n-1,0}x_0 + a_{n-1,1}x_1 + \cdots + a_{n-1,n-1}x_{n-1} = b_{n-1}$$

- Metoda lui Gauss constă în reducerea sistemului la forma triunghiulară

$$x_0 + a'_{0,1}x_1 + \cdots + a'_{0,n-1}x_{n-1} = b'_0$$

$$x_1 + \cdots + a'_{1,n-1}x_{n-1} = b'_1$$

.....

$$x_{n-1} = b'_{n-1}$$

și rezolvarea ecuațiilor în ordine inversă (întâi ultima, apoi penultima și la sfârșit prima ecuație din sistem)

Algoritmul eliminării Gaussiene – varianta secvențială

► *Notatii:*

- $A[0 \dots n-1, 0 \dots n-1]$ este un tablou bidimensional, de dimensiune $n \times n$.
- $B[0 \dots n-1]$ este un tablou unidimensional, de dimensiune n .

► *Premise:*

- Coeficienții inițiali $a_{i,j}, i, j = 0, 1, \dots, n-1$ sunt memorați în tabloul A .
- Coeficienții $a'_{i,j}, i, j = 0, 1, \dots, n-1$ vor fi memorați tot în tabloul A .
- Tabloul B va memora coeficienții inițiali $b_i, i = 0, 1, \dots, n-1$ și soluția sistemului.

Algoritmul eliminării Gaussiene – varianta secvențială

Algoritm 32: Varianta secvențială a eliminării Gaussiene

```
1: procedure ELIMINARE_GAUSSIANA( $A, B, n$ )  
2:   for  $k \leftarrow 0 \rightarrow n-1$  do ▷ bucla exterioara  
3:     for  $j \leftarrow k+1 \rightarrow n-1$  do  
4:        $A[k, j] \leftarrow \frac{A[k, j]}{A[k, k]}$  ▷ pasul de impartire cu  $A[k, k]$   
5:      $B[k] \leftarrow \frac{B[k]}{A[k, k]}$   
6:     for  $i \leftarrow k+1 \rightarrow n-1$  do  
7:       for  $j \leftarrow k+1 \rightarrow n-1$  do  
8:          $A[i, j] \leftarrow A[i, j] - A[i, k] \times A[k, j]$  ▷ pasul de eliminare  
9:        $B[i] \leftarrow B[i] - A[i, k] \times B[k]$   
10:       $A[i, k] \leftarrow 0$ 
```

Algoritmul eliminării Gaussiene – varianta paralelă

- Se mențin aceleași notații și premise ca în cazul secvențial

Algoritm 33: Varianta paralelă a eliminării Gaussiene

```

1: procedure ELIMINARE_GAUSSIANA_PARALELA( $A, B, n$ )
2:   for  $k \leftarrow 0 \rightarrow n-1$  do                                     ▷ bucla exterioara
3:     for  $j \leftarrow k+1 \rightarrow n-1$  do
4:        $A[k, j] \leftarrow \frac{A[k, j]}{A[k, k]}$                                ▷ pasul de impartire cu  $A[k, k]$ 
5:        $B[k] \leftarrow \frac{B[k]}{A[k, k]}$ 
6:       for  $i \leftarrow k+1 \rightarrow n-1$  parallel do
7:         for  $j \leftarrow k+1 \rightarrow n-1$  do
8:            $A[i, j] \leftarrow A[i, j] - A[i, k] \times A[k, j]$            ▷ pasul de eliminare
9:            $B[i] \leftarrow B[i] - A[i, k] \times B[k]$ 
10:           $A[i, k] \leftarrow 0$ 
11:       end for

```

Implementare pe un sistem cu n unități de procesare

- ▶ Se consideră un sistem format din n unități de procesare, $p_0, p_1, \dots, p_{n-1}$.
- ▶ Fiecare unitate de procesare $p_i, i \in \{0, 1, \dots, n-1\}$, dispune, în memoria locală, de coeficienții ecuației de indice i din sistem (linia i a tabloului A plus $B[i]$).
- ▶ În prima fază, unitatea de procesare p_0 execută pasul de împărțirea cu $A[0,0]$:

$$A[0,j] = \frac{A[0,j]}{A[0,0]}, j = 0, \dots, n-1.$$
 Apoi trimite $A[0,j], j = 0, 1, \dots, n-1$ celorlalte unități de procesare, după care, fiecare unitate de procesare $p_i, i = 1 \dots, n-1$ execută pasul de eliminare: $A[i,j] = A[i,j] - A[i,0] \times A[0,j], i, j = 1, \dots, n-1$ și $A[i,0] = 0, i = 1 \dots, n-1$.
- ▶ În faza a doua, p_1 preia rolul lui p_0 și execută împărțirea cu $A[1,1]$:

$$A[1,j] = \frac{A[1,j]}{A[1,1]}, j = 1 \dots, n-1.$$
 Apoi trimite $A[1,j], j = 1 \dots, n-1$ unităților de procesare $p_2, \dots, p_{n-1}$, după care, fiecare unitate de procesare $p_i, i = 2 \dots, n-1$ execută pasul de eliminare: $A[i,j] = A[i,j] - A[i,1] \times A[1,j], i, j = 2, \dots, n-1$ și $A[i,1] = 0, i = 2, \dots, n-1$.
- ▶ În faza k , p_k execută împărțirea cu $A[k,k]$: $A[k,j] = \frac{A[k,j]}{A[k,k]}, j = k \dots, n-1$. Apoi trimite $A[k,j], j = k \dots, n-1$ unităților de procesare $p_{k+1}, \dots, p_{n-1}$, după care, fiecare unitate de procesare $p_i, i = k+1 \dots, n-1$ execută pasul de eliminare: $A[i,j] = A[i,j] - A[i,k] \times A[k,j], i, j = k+1, \dots, n-1$ și $A[i,k] = 0, i = k+1, \dots, n-1$.

Exemplu de implementare pe un sistem cu 8 unități de procesare

P_0	1	(0,1)	(0,2)	(0,3)	(0,4)	(0,5)	(0,6)	(0,7)
P_1	0	1	(1,2)	(1,3)	(1,4)	(1,5)	(1,6)	(1,7)
P_2	0	0	1	(2,3)	(2,4)	(2,5)	(2,6)	(2,7)
P_3	0	0	0	(3,3)	(3,4)	(3,5)	(3,6)	(3,7)
P_4	0	0	0	(4,3)	(4,4)	(4,5)	(4,6)	(4,7)
P_5	0	0	0	(5,3)	(5,4)	(5,5)	(5,6)	(5,7)
P_6	0	0	0	(6,3)	(6,4)	(6,5)	(6,6)	(6,7)
P_7	0	0	0	(7,3)	(7,4)	(7,5)	(7,6)	(7,7)

Figura 54: Pasul de împărțire cu $A[3,3]$:

$$A[3,j] = \frac{A[3,j]}{A[3,3]}, j = 4, \dots, 7, \text{ și } A[3,3] = 1$$

Exemplu de implementare pe un sistem cu 8 unități de procesare – continuare

P_0	1	(0,1)	(0,2)	(0,3)	(0,4)	(0,5)	(0,6)	(0,7)
P_1	0	1	(1,2)	(1,3)	(1,4)	(1,5)	(1,6)	(1,7)
P_2	0	0	1	(2,3)	(2,4)	(2,5)	(2,6)	(2,7)
P_3	0	0	0	1	(3,4)	(3,5)	(3,6)	(3,7)
P_4	0	0	0	(4,3)	(4,4)	(4,5)	(4,6)	(4,7)
P_5	0	0	0	(5,3)	(5,4)	(5,5)	(5,6)	(5,7)
P_6	0	0	0	(6,3)	(6,4)	(6,5)	(6,6)	(6,7)
P_7	0	0	0	(7,3)	(7,4)	(7,5)	(7,6)	(7,7)

Figura 55: Broadcast linia 3: $A[3,j], j = 4, \dots, 7$

Exemplu de implementare pe un sistem cu 8 unități de procesare – continuare

P_0	1	(0,1)	(0,2)	(0,3)	(0,4)	(0,5)	(0,6)	(0,7)
P_1	0	1	(1,2)	(1,3)	(1,4)	(1,5)	(1,6)	(1,7)
P_2	0	0	1	(2,3)	(2,4)	(2,5)	(2,6)	(2,7)
P_3	0	0	0	1	(3,4)	(3,5)	(3,6)	(3,7)
P_4	0	0	0	(4,3)	(4,4)	(4,5)	(4,6)	(4,7)
P_5	0	0	0	(5,3)	(5,4)	(5,5)	(5,6)	(5,7)
P_6	0	0	0	(6,3)	(6,4)	(6,5)	(6,6)	(6,7)
P_7	0	0	0	(7,3)	(7,4)	(7,5)	(7,6)	(7,7)

Figura 56: Pasul de eliminare:

$$A[i,j] = A[i,j] - A[i,3] \times A[3,j], i, j = 4, \dots, 7 \text{ și}$$

$$A[i,3] = 0, i = 4, \dots, 7$$

Complexitatea timp a implementării algoritmului eliminării Gaussiene pe un lanț de n unități de procesare

Teorema (1)

Complexitatea timp a implementării algoritmului eliminării Gaussiene pe un lanț de n unități de procesare este $\Theta(n^2)$.

Demonstrație.

Timpul consumat în iterația k este $\Theta(n - k - 1)$. Timpul total este

$$\sum_{k=0}^{n-1} \Theta(n - k - 1) = \Theta(n - 1) + \Theta(n - 2) + \cdots + \Theta(0) = \Theta(n^2).$$



Exercițiu

Rezolvarea completă a sistemului de ecuații

Pornind de la Algoritmul 33, descrieți un pseudocod pentru etapa de determinare a soluțiilor unui sistem de ecuații liniare cu n ecuații și n necunoscute.

Comentarii bibliografice

- Capitolul *Sisteme de ecuații liniare* are la bază cartea
V. Kumar, A. Grama, A. Gupta & G Karypis, Introduction to Parallel Computing: Design and Analysis of Algorithms, Addison Wesley, 2003
și ediția mai veche
V. Kumar, A. Grama, A. Gupta & G Karypis, Introduction to Parallel Computing: Design and Analysis of Algorithms, Benjamin-Cummings, 1994

Cuprins

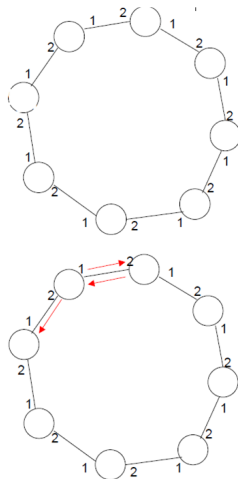
1. Introducere
2. Modele de comunicare
3. Topologii de comunicare
4. Modele de calcul paralel/distribuit
5. Parametri de performanță a unui algoritm paralel/distribuit
6. Comunicarea în sistemele de calcul paralel/distribuit
7. Algoritmi fundamentali
8. Sortare paralelă
9. Calcul matriceal
10. Sisteme de ecuații liniare
- 11. Alegerea liderului**
12. Excluderea mutuală în sisteme *shared-memory*

Introducere

- ▶ Problemă fundamentală: în sistemele distribuite este frecvent nevoie de un coordonator.
- ▶ Aplicații diverse:
 - ▶ defecțiuni ale componentelor unui sistem distribuit (de exemplu: alegerea unui nou server coordonator pentru continuarea funcționării unui serviciu);
 - ▶ excludere mutuală în sisteme bazate pe comunicarea prin mesaje;
 - ▶ rețele mobile – alegerea unui alt lider când liderul cunoscut părăsește rețeaua.
- ▶ Topologii de comunicare:
 - ▶ inelul;
 - ▶ graful.

Alegerea liderului în rețele cu topologie de comunicare inel – aspecte generale

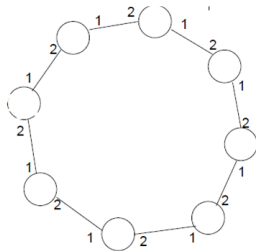
- ▶ Fiecare nod al inelului este asociat unei unități de procesare.
- ▶ Se stabilesc sensuri în inel. De exemplu, stânga și dreapta unui nod este fixată din perspectiva așezării nodului cu "fața" la centru.
- ▶ Legăturile între noduri pot fi bidirecționale.



Aspecte generale – continuare

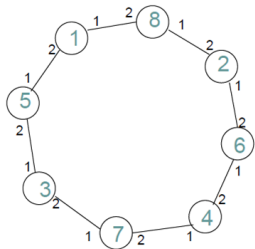
► Inel anonim:

- Unitățile de procesare nu au identificatori.
- Au mecanisme identice pentru schimbarea stărilor, adică funcțiile de tranziție de stare sunt identice.
- Sunt exprimate în termeni de stânga și dreapta.



► Inel etichetat:

- Fiecare unitate de procesare p_i , $i \in \{0, 1, \dots, n-1\}$ are asociat un identificator unic id_i
- Funcțiile de tranziție de stare sunt diferite.
- Sunt identificate prin id -urile lor.



Aspecte generale – continuare

► Inel uniform:

- Unitățile de procesare au identificatori unici.
- Numărul nodurilor inelului (n) nu este cunoscut de către unitățile de procesare.
- Mecanismele de schimbare a stărilor nu depind de n (Funcțiile de tranziție de stare nu au variabila n ca parametru).

► Inel neuniform:

- Unitățile de procesare au identificatori unici.
- Numărul nodurilor inelului (n) este cunoscut de către unitățile de procesare.
- Mecanismele de schimbare a stărilor depind de n (Funcțiile de tranziție de stare au variabila n ca parametru).

Aspecte generale – continuare

- ▶ Algoritmii de alegere a liderului depind de:
 - ▶ tipul inelului: anonim sau etichetat;
 - ▶ numărul nodurilor inelului: cunoscut sau necunoscut;
 - ▶ tipul de ceas: global (execuție sincronă) sau local (execuție asincronă).

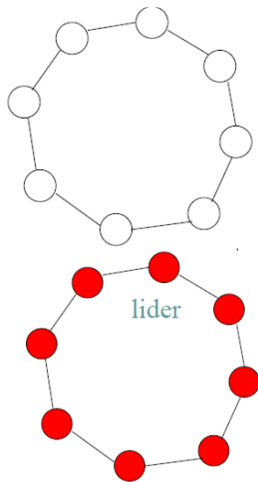
Alegerea liderului în inele anonime

► Inele anonime sincrone:

- Ceasul este global.
- Unitățile de procesare execută același program.
- La un moment dat, unitățile de procesare efectuează aceeași operație (execută aceeași instrucțiune)
- Unitățile de procesare au aceeași evoluție, deoarece sunt identice.
- La final, toate unitățile de procesare sunt lideri.

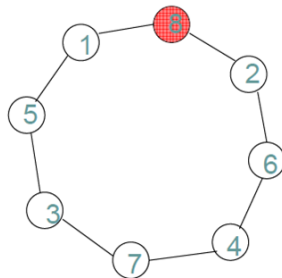
► Inele anonime asincrone:

- Ceasul este local.
- Unitățile de procesare au aceeași evoluție, deoarece sunt identice.
- La final, toate unitățile de procesare sunt lideri.



Alegerea liderului în inele etichetate

- Unitatea de procesare cu cel mai mare/mic identificator (*id*) este aleasă lider



Algoritm sincron de alegere a liderului în inele etichetate uniforme – descriere

- ▶ Startul nu este sincronizat. Aceasta înseamnă că unitățile de procesare se „trezesc” spontan sau după primirea unui mesaj, adică încep execuția programului de alegerea a liderului în momente diferite.
- ▶ La trezire, dacă nu a primit nici un mesaj, unitatea de procesare devine *participant*; dacă a primit un mesaj, devine *releu*.
- ▶ Atunci când o unitate de procesare cu rol de *participant* primește un mesaj de la p_i , dacă *id*-ul primit este mai mare decât identificatorul cel mai mic cunoscut (inclusiv propriul identificator), îl ignoră; altfel, îl reține $2^{id} - 1$ runde.
- ▶ Atunci când o unitate de procesare cu rol de *releu* primește un mesaj de la p_i , dacă *id*-ul primit este mai mare decât identificatorul cel mai mic cunoscut (fără propriul identificator), îl ignoră; altfel, îl retransmite imediat.

Algoritm sincron de alegere a liderului în inele etichetate uniforme

Pseudocod pentru unitatea de procesare p_i

► Notății:

- R_i memorează mesajele recepționate de unitatea de procesare p_i ;
- S_i memorează mesajele care urmează a fi trimise de către unitatea de procesare p_i ;
- W_i memorează mesajele întârziate (în așteptare);
- $stare_i$ reprezintă starea curentă a unității de procesare p_i (*adormit*, *participant* sau *releu*);
- $statut_i$ memorează starea finală a unității de procesare p_i (*lider* sau *non_lider*);
- h reprezintă faza în care se află mesajul m .

► Premise:

- Inițial $S_i = R_i = W_i = \emptyset$ și $stare_i = adormit$, $statut_i = nedefinit$.

Algoritm sincron de alegere a liderului în inele etichetate uniforme

Pseudocod pentru unitatea de procesare p_i

Algoritm 34: Algoritmul ASAL_IEU

```

1: procedure ASAL_IEU( $S_i, R_i, W_i, n, stare_i, statut_i, id_i, p_i$ )
2:   while true do
3:     if  $stare_i = adormit$  then
4:       if  $R_i = \emptyset$  then
5:          $stare_i \leftarrow participant$ 
6:          $min \leftarrow id_i$ 
7:         memoreaza ( $id_i, 1$ ) in  $S_i$ 
8:       else
9:          $stare_i \leftarrow releu$ 
10:         $min \leftarrow \infty$ 

```

Algoritm sincron de alegere a liderului în inele etichetate uniforme

Pseudocod pentru unitatea de procesare p_i (continuare)

Algoritm 34: Algoritmul ASAL_IEU (continuare)

```
11:   for each  $(m, h) \in R_i$  do
12:     if  $h \neq 3$  then
13:       if  $m < \min$  then
14:          $\min \leftarrow m$ 
15:       if  $stare_i = \text{releu}$  and  $h = 1$  then
16:         memoreaza  $(m, 1)$  in  $S_i$ 
17:       else
18:         memoreaza  $(m, 2)$  impreuna cu numarul rundei in  $W_i$ 
19:     else
20:       if  $m = id_i$  then
21:          $statut_i \leftarrow \text{lider}$ 
22:         memoreaza  $(m, 3)$  in  $S_i$ 
```

Algoritm sincron de alegere a liderului în inele etichetate uniforme

Pseudocod pentru unitatea de procesare p_i (continuare)

Algoritm 34: Algoritm ASAL_I EU (continuare)

```

23:         else
24:             if  $statut_i = lider$  then
25:                 break                                ▷ termina executia algoritmului
26:             else
27:                  $statut_i \leftarrow non\_lider$ 
28:                 memoreaza  $(m, 3)$  in  $S_i$ 
29:         for each  $(m, 2) \in W_i$  do
30:             if  $(m, 2)$  a fost primit in urma cu  $2^m - 1$  runde then
31:                 sterge  $(m, 2)$  din  $W_i$  si memoreaza in  $S_i$ 
32:             trimite  $S_i$  spre stanga
33:             if  $statut_i = non\_lider$  then
34:                 break                                ▷ termina executia algoritmului

```

Algoritm sincron de alegere a liderului în inele etichetate uniforme – corectitudinea

Teorema (1)

Doar unitatea de procesare cu cel mai mic id primește înapoi propriul id.

Demonstrație.

Fie p_i unitatea de procesare participantă cu cel mai mic identificator (id_i). Cel puțin o unitate de procesare are statut de participant. Evident, nici o unitate de procesare nu poate „înghiți” (fără a-l transmite mai departe) mesajul id_i . Mai mult, mesajul id_i este întârziat, în fiecare unitate de procesare, cel mult 2^{id_i} runde. Să presupunem că există o unitate de procesare p_j , $j \neq i$, care primește înapoi propriul identificator, id_j . Rezulă că id_j trece prin toate unitățile de procesare din inel, inclusiv prin p_i . Dar $id_i < id_j$ și p_i este o unitate de procesare participantă, care nu va retransmite id_j . Contradicție. □

Algoritm sincron de alegere a liderului în inele etichetate uniforme – complexitatea

Mesajele care circulă prin inel pot fi clasificate în patru categorii:

1. mesaje în faza I ($h = 1$);
2. mesaje în faza II ($h = 2$) trimise înainte de intrarea mesajului liderului în faza II;
3. mesaje în faza II ($h = 2$) trimise după intrarea mesajului liderului în faza II.
4. mesaje în faza III ($h = 3$) trimise după ce a fost decis liderul.

Lema (1)

Numărul de mesaje din prima categorie este cel mult n .

Demonstrație.

Este suficient să demonstrăm că fiecare unitate de procesare retransmite cel mult un mesaj din prima categorie. Să presupunem că există o unitate de procesare p_i care retransmite două mesaje din prima categorie, id_j primit de la p_j și id_k de la p_k . Fără a restrânge generalitatea, presupunem că p_j este mai aproape de p_i decât p_k . Dacă id_k ajunge în p_j după ce p_j se „trezește” și trimite p_j , id_k trece în faza II și va fi întârziat în fiecare unitate de procesare participantă 2^{id_k} runde; altfel p_j nu participă și nu trimite identificatorul id_j . Prin urmare, fie id_k ajunge în p_i în faza II, fie id_j nu este trimis de p_j și prin urmare nu este primit de p_i . Contradicție. □

Algoritm sincron de alegere a liderului în inele etichetate uniforme – complexitatea

Lema (2)

Fie r prima rundă în care se „trezește” măcar o unitate de procesare și începe execuția algoritmului. Fie p_i una dintre acestea. Dacă unitatea de procesare p_j este la distanță d față de p_i , atunci p_j primește un mesaj din prima categorie cel mult în runda $r + d$.

Demonstrație.

Inducție după d . Pentru $d = 1$ este evident că vecinul lui p_i primește mesajul id_i (din categoria 1) în runda $r + 1$. Să presupunem că unitatea de procesare p_k situată la distanța $d - 1$ față de p_i primește un mesaj din prima categorie, cel mult în runda $r + d - 1$. Dacă aceasta este „trezită”, a trimis deja la vecinul p_j un mesaj din prima categorie; altfel, înseamnă că p_k are statut de *releu* și va retransmite în runda $r + d$ către p_j mesajul de categoria 1, primit anterior. $\square$

Algoritm sincron de alegere a liderului în inele etichetate uniforme – complexitatea

Lema (3)

Numărul de mesaje din categoria a doua este cel mult n .

Demonstrație.

Din Lema 1 rezultă că fiecare unitate de procesare retransmite cel mult un mesaj din prima categorie, adică pe o muchie este transmis cel mult un mesaj din prima categorie. Din Lema 2 rezultă că în runda $r + n$ pe fiecare muchie a fost trimis un mesaj din categoria 1. Rezultă că după runda $r + n$ nu va mai fi transmis nici un mesaj din categoria 1. Din Lema 2 rezultă că mesajul liderului intră în faza a II-a după cel mult $r + n$ runde, adică nu mai târziu de a n -a rundă care urmează trezirii primei unități de procesare. Rezultă că mesajele din categoria 2 circulă cel mult n runde. Un mesaj m aflat în faza a II-a este întârziat $2^m - 1$ runde, după care este retransmis. Prin urmare, un mesaj m circulă cu statutul „categoria 2” cel mult de $\frac{n}{2^m}$ ori. Deoarece mesajul care conține cel mai mic identificator circulă de cele mai multe ori, rezultă că numărul maxim de mesaje se obține atunci când unitățile de procesare participante au identificatorii $0, 1, \dots, n-1$. Dacă ținem cont că mesajul liderului nu poate face parte din categoria 2, rezultă că numărul mesajelor din categoria 2 este $\sum_{i=1}^{n-1} \frac{n}{2^i} \leq n$



Algoritm sincron de alegere a liderului în inele etichetate uniforme – complexitate

Lema (4)

Nici un mesaj nu este retransmis după ce liderul p_i primește propriul identificator id_i .

Demonstrație.

Toate mesajele care urmează lui id_i sunt „înghițite”.



Algoritm sincron de alegere a liderului în inele etichetate uniforme – complexitate

Lema (5)

Numărul de mesaje din categoria a treia este cel mult $2n$.

Demonstrație.

Fie p_i liderul și p_j o unitate de procesare cu statut de *participant*. Din Teorema 1 rezultă că $id_i < id_j$. Din Lema 4 rezultă că nici un mesaj nu este retransmis după ce p_i primește înapoi propriul identificator id_i . Deoarece id_j este întârziat cel mult 2^{id_i} runde în fiecare unitate de procesare, sunt necesare cel mult $n2^{id_i}$ runde pentru ca p_i să primească înapoi propriul identificator id_i . Așadar, mesajele din categoria a treia sunt transmise de-a lungul a cel mult $n2^{id_i}$ runde. În timpul acestor runde, id_j este retransmis cel mult de $\frac{1}{2^{id_j}} n2^{id_i} = \frac{n}{2^{id_j-id_i}}$ ori. Astfel, numărul mesajelor din categoria 3 este cel mult $\sum_{j=0}^{n-1} \frac{n}{2^{id_j-id_i}}$. Pe baza faptului că numărul maxim de mesaje se obține atunci când unitățile de procesare participante au identificatorii $0, 1, \dots, n-1$ (Lema 3) se deduce că $\sum_{j=0}^{n-1} \frac{n}{2^{id_j-id_i}} = \sum_{k=0}^{n-1} \frac{n}{2^k} < 2n$. □

Algoritm sincron de alegere a liderului în inele etichetate uniforme – complexitate

Lema (6)

Numărul de mesaje din categoria a patra este n .

Demonstrație.

Fiecare unitatea de procesare transmite mai departe mesajul liderului ($h = 3$), prin care acesta se declară câștigător. □

Algoritm sincron de alegere a liderului în inele etichetate uniforme – complexitate

Teorema (2)

Numărul de mesaje circulate prin algoritmul sincron de alegere a liderului în inele etichetate uniforme este cel mult $5n$.

Demonstrație.

Consecință imediată a lemelor 1,3,5 și 6.



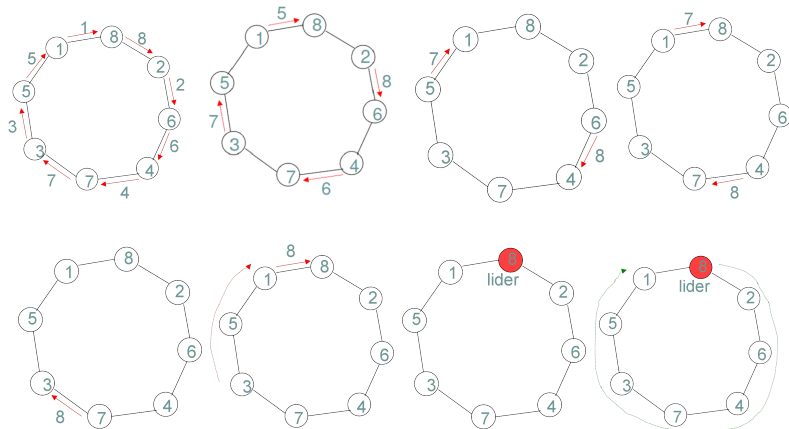
Algoritm sincron de alegere a liderului în inele etichetate neuniforme – descriere

- ▶ Algoritmul se desfășoară în faze. Fiecare fază este compusă din n runde.
- ▶ Startul este sincronizat. Aceasta înseamnă că toate unitățile de procesare se „trezesc” simultan, adică încep execuția programului de alegerea a liderului în același moment.
- ▶ Unitatea de procesare cu cel mai mic identificator este aleasă lider.
- ▶ În fiecare rundă, fiecare unitate de procesare efectuează următoarele operații:
 - ▶ analizează mesaje primite pe canalele din stânga și din dreapta;
 - ▶ își schimbă starea în funcție de starea curentă și de mesajele primite; dacă nu a primit niciun mesaj, își schimbă starea doar în funcție de starea curentă;
 - ▶ transmite mesaje la vecini, dacă are ceva de transmis.
- ▶ În faza i , dacă nimeni nu este ales, atunci unitatea de procesare cu id -ul i se autodeclară lider și transmite la vecini un mesaj prin care se declară lider; celelalte unități de procesare retransmit mesajul primit.

Algoritmul asincron LCR (LeLann, Chang și Roberts) – descriere

- ▶ Se consideră că unitățile de procesare „stau cu fața” spre centrul inelului.
- ▶ Fiecare unitate de procesare p_i transmite identificatorul său (id_i) vecinului din stânga.
- ▶ Apoi, așteaptă răspuns de la vecinul din dreapta
- ▶ Dacă identificatorul recepționat este mai mare decât propriul identificator, transmite identificatorul primit spre stânga.
- ▶ Dacă identificatorul primit e mai mic decât propriul identificator, ignoră („inghite”) mesajul.
- ▶ Dacă identificatorul recepționat este propriul identificator, unitatea respectivă se declară *lider* și transmite în inel un mesaj de terminare.
- ▶ Dacă o unitate de procesare primește mesaj de terminare, termină ca *non-lider*.

Algoritmul asincron LCR (LeLann, Chang și Roberts) – exemplu de execuție



Algoritmul asincron LCR (LeLann, Chang și Roberts) – pseudocod

Pseudocod pentru unitatea de procesare p_i

► *Notatii:*

- id_i reprezintă identificatorul unității de procesare p_i .
- $stare_i$ memorează starea curentă a unității de procesare p_i (*adormit*, *trezit*).
- $statut_i$ memorează starea finală a unității de procesare p_i (*lider* sau *non_lider*)

► *Premise:*

- Inițial $stare_i = adormit$ și $statut_i = necunoscut$.

Algoritmul asincron LCR (LeLann, Chang și Roberts) – pseudocod

Pseudocod pentru unitatea de procesare p_i

Algoritm 35: Algoritmul LCR

```
1: function LCR_LE( $p_i, stare_i, statut_i, lider$ )
2:    $stare_i \leftarrow trezit$ 
3:   trimite  $\langle "id\_nou", id_i \rangle$  spre stanga
4:   while true do
5:     switch
6:       case primește  $\langle "id\_nou", id \rangle$  dinspre dreapta
7:         if  $id = id_i$  then
8:            $statut_i \leftarrow lider$ 
9:            $lider \leftarrow id_i$ 
10:          trimite  $\langle "lider", lider \rangle$  spre stanga
11:          break while
12:        if  $id > id_i$  then
13:          trimite  $\langle "id\_nou", id \rangle$  spre stanga
14:        break
```

Algoritmul asincron LCR (LeLann, Chang și Roberts) – pseudocod

Pseudocod pentru unitatea de procesare p_i (continuare)

Algoritm 35: Algoritmul LCR (continuare)

```
15:         case primește  $\langle "lider", id \rangle$  dinspre dreapta
16:              $statut_i \leftarrow non\_lider$ 
17:              $lider \leftarrow id$ 
18:             trimite  $\langle "lider", lider \rangle$  spre stanga
19:             break while
20:         end switch
```

Algoritmul asincron LCR (LeLann, Chang și Roberts) – corectitudinea

Teorema (3)

Doar unitatea de procesare cu cel mai mare id primește primește înapoi propriul id.

Demonstrație.

Doar mesajul unității de procesare cu cel mai mare *id* nu va fi „înghițit” (ignorat).



Algoritmul asincron LCR (LeLann, Chang și Roberts) – complexitatea

Teorema (4)

Numărul de mesaje care circulă prin inel este cel mult n^2 .

Demonstrație.

Cazul cel mai nefavorabil este acela în care unitățile de procesare au identificatori din mulțimea $\{0, 1, \dots, n-1\}$ și sunt plasate pe inel în ordinea $n-1, n-2, \dots, 0$. În această situație, mesajul $id_i = i$ al unității de procesare p_i este retransmis exact de $i+1$ ori. Astfel, numărul mesajelor care circulă prin inel (inclusiv mesajul de terminare) este $n + \sum_{i=0}^{n-1} (i+1) = O(n^2)$. □

Algoritmul FloodMax sincron – descriere

- ▶ Graful $G = (V, E)$ este conex. Unitățile de procesare cunosc diametrul $= d$.
- ▶ Fiecare unitate de procesare conține o înregistrare a identificatorului maxim cunoscut; inițial acesta este propriul identificator.
- ▶ La fiecare rundă, fiecare unitate de procesare trimite acest maxim vecinilor.
- ▶ După d runde, dacă maximum cunoscut este propriul identificator, atunci unitatea de procesare se declară *lider*; altfel se consideră *non-lider*.

Algoritmul FloodMax sincron – pseudocod

Pseudocod pentru unitatea de procesare p_i

► *Notatii:*

- id_i reprezintă identificatorul unității de procesare p_i .
- R_i memorează mesajele recepționate de unitatea de procesare p_i într-o rundă.
- max_id_i memorează cel mai mare identificator cunoscut de p_i .
- $statut_i$ memorează starea finală a unității de procesare p_i (*lider* sau *non_lider*).

► *Premise:*

- Inițial $max_id_i = id_i$ și $statut_i = necunoscut$.

Algoritmul FloodMax sincron – pseudocod

Pseudocod pentru unitatea de procesare p_i

Algoritm 36: Algoritmul FloodMax sincron

```
1: function FLOODMAX_LE( $G, d, p_i, max\_id_i, statut_i$ )
2:   for  $runda \leftarrow 1$  to  $d$  do
3:     for each  $m \in R_i$  do
4:       if  $max\_id_i < m$  then
5:          $max\_id_i \leftarrow m$ 
6:       trimite  $max\_id_i$  tuturor vecinilor
7:   if  $max\_id_i = id_i$  then
8:      $statut_i \leftarrow lider$ 
9:   else
10:     $statut_i \leftarrow non - lider$ 
```

Algoritmul FloodMax sincron – complexitatea

Teorema (5)

Numărul de mesaje care circulă prin graf este $2d|E|$.

Demonstrație.

Într-o rundă, pe fiecare muchie circulă două mesaje. Numărul rundelor este d .



Cuprins

1. Introducere
2. Modele de comunicare
3. Topologii de comunicare
4. Modele de calcul paralel/distribuit
5. Parametri de performanță a unui algoritm paralel/distribuit
6. Comunicarea în sistemele de calcul paralel/distribuit
7. Algoritmi fundamentali
8. Sortare paralelă
9. Calcul matriceal
10. Sisteme de ecuații liniare
11. Alegerea liderului
12. Excluderea mutuală în sisteme *shared-memory*

Excluderea mutuală în sisteme *shared-memory* – introducere

- ▶ Problemă fundamentală: în sistemele distribuite este frecvent nevoie de a arbitra accesul concurent la o resursă.
- ▶ Problema excluderii mutuale apare la un grup de unități de procesare care periodic accesează anumite resurse care nu pot fi utilizate simultan de mai mult de o singură unitate de procesare (de exemplu, o imprimantă).
- ▶ Fiecare unitate de procesare poate dori să execute un segment de cod numit secțiune critică, astfel încât, la orice moment, cel mult o unitate de procesare este în secțiunea critică (*excluderea mutuală*).
- ▶ *Dacă una sau mai multe unități de procesare încearcă să intre în secțiunea critică, atunci una dintre ele va reuși în cele din urmă, atâta timp cât nici o unitate de procesare nu stă la infinit în secțiunea critică.*
- ▶ Proprietatea de mai sus nu dă nici o garanție de reușită, deoarece o unitate de procesare poate încerca să intre în secțiunea critică și totuși să nu reușească, aceasta fiind ignorată mereu de alte unități de procesare.
- ▶ O proprietate mai puternică, care elimină *blocarea (deadlock)*, este *fără flămânzire (no lockout sau no starvation)*: *Dacă o unitate de procesare dorește să intre în secțiunea critică, aceasta va reuși în cele din urmă, atâta timp cât nici o unitate de procesare nu rămâne la infinit în secțiunea critică.*

Excluderea mutuală în sisteme *shared-memory* – introducere (2)

- ▶ Programul unei unități de procesare este împărțit în următoarele secțiuni:
 - Entry:** codul executat la pregătirea pentru intrarea în secțiunea critică;
 - Critical:** codul care trebuie protejat de execuție concurentă;
 - Exit:** codul executat după părăsirea secțiunii critice;
 - Remainder:** restul codului.

Excluderea mutuală cu regiștri binari *test&set* – descriere

Un registru *test&set* memorează valori binare și suportă două operații atomice, *test&set* și *reset*, definite de Algoritmii 37 și 38.

► *Notatii:*

- r este un registru *test&set*.

Algoritm 37: Operația atomică *test&set*

```
1: procedure TEST & SET( $r$ )  
2:    $temp \leftarrow r$   
3:    $r \leftarrow 1$   
4:   return  $temp$ 
```

Algoritm 38: Operația atomică *reset*

```
1: procedure RESET( $r$ )  
2:    $r \leftarrow 0$ 
```

Excluderea mutuală cu regiștri binari *test&set* – descriere (2)

- ▶ Se presupune că valoarea inițială a registrului r este 0.
- ▶ În secțiunea de intrare, unitatea de procesare p_i testează r în mod repetat, până când testul returnează 0; după ultimul test r va conține 1.
- ▶ Orice test ulterior va returna 1, ceea ce interzice oricărei alte unități de procesare să intre în secțiunea critică.
- ▶ În secțiunea de ieșire, p_i resetează $r = 0$, astfel încât una dintre unitățile de procesare, care așteaptă în secțiunea de intrare, să poată intra în secțiunea critică.
- ▶ Excluderea mutuală realizată prin acest algoritm este fără blocaj (*no deadlock*), dar cu flămânzire (*lockout*).

Excluderea mutuală cu regiștri binari *test&set* – pseudocod

- ▶ *Notății:*
 - ▶ r este un registru *test&set*.
- ▶ *Premise:* Inițial $r = 0$.

Algoritm 39: Pseudocod pentru unitatea de procesare $p_i, i \in \{0, 1, \dots, n-1\}$

⟨Entry⟩ / p_i are nevoie de resursa critică*/*

- 1: **procedure** ENTRY(r)
- 2: *asteapta pana cand* TEST & SET(r) = 0

⟨Critical⟩ / p_i intră în regiunea critică*/*

⟨Exit⟩ / p_i părăsește regiunea critică*/*

- 1: **procedure** EXIT(r)
- 2: RESET(r)

⟨Remainder⟩ / p_i nu mai are nevoie de resursa critică*/*

Excluderea mutuală cu regiștri *read-modify-write* – descriere

- ▶ Un registru r , de tipul *read-modify-write*, are următoarele proprietăți:
 - ▶ Valorile v memorate de registrul r sunt numere întregi.
 - ▶ Operația suportată este $RMW(r, f)$, unde f este o funcție definită pe $\mathbb{Z}$ cu valori în $\mathbb{Z}$, adică $f : \mathbb{Z} \leftarrow \mathbb{Z}$.
 - ▶ Operația $RMW(r, f)$ este atomică, întoarce valoarea v memorată în registrul r și schimbă conținutul lui r , înscriind acolo valoarea $f(v)$.
- ▶ Operația *test&set* este un caz particular al operației RMW , în care $f(r) = 1$, pentru orice valoare a lui r .

Algoritm 40: Operația atomică RMW

```
1: procedure  $RMW(r, f)$   
2:    $v \leftarrow r$   
3:    $r \leftarrow f(v)$   
4:   return  $v$ 
```

Excluderea mutuală cu regiștri *read-modify-write* – descriere (2)

- ▶ Unitățile de procesare sunt organizate într-o coadă de tip *FIFO*
- ▶ Numai unitatea de procesare de la începutul cozii poate intra în secțiunea critică și rămâne la începutul cozii până părăsește secțiunea critică, nepermițând în acest fel celorlalte unități de procesare să intre în secțiunea critică.
- ▶ Excluderea mutuală realizată prin acest algoritm este fără blocaj (*no deadlock*) și fără flămânzire (*no lockout*).

Excluderea mutuală cu regiștri *read-modify-write* - descriere (3)

- ▶ Fiecare unitate de procesare dispune de două variabile locale, *pozitie* și *coada*.
- ▶ Algoritmul utilizează un registru *r* de tip *read-modify-write*, care memorează o structură formată din două câmpuri: *primul* și *ultimul*.
- ▶ *r.primul* și *r.ultimul* conțin „bilete” pentru prima și respectiv pentru ultima unitate de procesare din coadă.
- ▶ Atunci când o unitate de procesare ajunge în secțiunea de intrare, se „așează” la coadă, prin asignarea valorii lui *r* unei variabile locale și prin incrementarea lui *r.ultimul*. Toate acestea sunt realizate printr-o singură operație, indivizibilă.
- ▶ Valoarea curentă a lui *r.ultimul* îndeplinește rolul de "bilet" pentru unitatea de procesare.
- ▶ O unitate de procesare așteaptă în secțiunea de intrare până devine prima din coadă, adică până când *r.primul* este egal cu biletul său. În acest moment, unitatea de procesare intră în secțiunea critică.
- ▶ După părăsirea secțiunii critice, unitatea de procesare se autoelimină din coadă prin incrementarea lui *r.primul*, permițând în acest fel următoarei unități de procesare din coadă să intre în secțiunea critică.

Excluderea mutuală cu regiștri *read-modify-write* – pseudocod

- ▶ *Notatii:*
 - ▶ r este un registru de tipul *read-modify-write*
- ▶ *Premise:* Inițial, $r = (0,0)$

Algoritm 41: Pseudocod pentru unitatea de procesare $p_i, i \in \{0, 1, \dots, n-1\}$

<Entry> / p_i are nevoie de resursa critică */*

- 1: **procedure** ENTRY(r)
- 2: $\text{pozitie} \leftarrow \text{RMW}(r, \langle r.\text{ultimul}, r.\text{ultimul} + 1 \rangle)$ ▷ se „așează” la coadă
- 3: **repeat**
- 4: $\text{coada} \leftarrow \text{RMW}(r, r)$ ▷ „citește” coada
- 5: **until** $\text{coada.primul} = \text{pozitie.ultimul}$ ▷ până când ajunge prima

<Critical> / p_i intră în regiunea critică */*

<Exit> / p_i părăsește regiunea critică */*

- 1: **procedure** EXIT(r)
- 2: $\text{RMW}(r, \langle r.\text{primul} + 1, r.\text{ultimul} \rangle)$

<Remainder> / p_i nu mai are nevoie de resursa critică */*

Excludere mutuală cu regiștri cu valori mărginite, pentru două unități de procesare, cu flămânzire – descriere

- ▶ Fiecare unitate de procesare p_i dispune de o variabilă booleană $vreau[i]$ partajată, a cărei valoare este 1, dacă p_i este interesată să intre în secțiunea critică și 0 în caz contrar.
- ▶ Algoritmul este asimetric:
 - ▶ p_0 intră în secțiunea critică fără să țină seama de încercările lui p_1 de a face același lucru;
 - ▶ p_1 intră în secțiunea critică numai dacă p_0 nu este interesat de aceasta.
- ▶ Acest algoritm utilizează un mecanism cu fanioane pentru coordonarea unităților de procesare care își dispută secțiunea critică:
 - ▶ p_i ridică un fanion (prin setarea $vreau[i] \leftarrow 1$) și apoi urmărește fanionul celeilalte unități de procesare (prin citirea lui $vreau[1 - i]$).
 - ▶ Cel puțin una dintre unitățile de procesare observă fanionul ridicat al celeilalte unități de procesare și evită intrarea în secțiunea critică (excludere mutuală).
- ▶ Dacă p_0 este interesată continuu să intre în secțiunea critică, este posibil ca p_1 să nu intre niciodată în secțiunea critică, având în vedere faptul că aceasta cedează de fiecare dată (*lockout*).

Excludere mutuală cu regiștri cu valori mărginite, pentru două unități de procesare, cu flămânzire – pseudocod

► *Premise:* Inițial, $vreau[0] = vreau[1] = 0$

Algoritm 42: Pseudocod pentru unitatea de procesare p_0

⟨Entry⟩

```
1: procedure ENTRY(vreau)  
2:   vreau[0] ← 1  
3:   asteapta pana cand vreau[1] = 0
```

⟨Critical⟩

⟨Exit⟩

```
1: procedure EXIT(vreau)  
2:   vreau[0] ← 0
```

⟨Remainder⟩

Algoritm 43: Pseudocod pentru unitatea de procesare p_1

⟨Entry⟩

```
1: procedure ENTRY(vreau)  
2:   vreau[1] ← 0  
3:   asteapta pana cand vreau[0] = 0  
4:   vreau[1] ← 1  
5:   if vreau[0] = 1 then  
6:     go to linia 2
```

⟨Critical⟩

⟨Exit⟩

```
1: procedure EXIT(vreau)  
2:   vreau[1] ← 0
```

⟨Remainder⟩

Excludere mutuală cu regiștri cu valori mărginite, pentru două unități de procesare, fără flămânzire – descriere

- ▶ Pentru a obține *no lockout* se modifică algoritmul precedent, astfel încât p_0 să nu mai fie prioritară continuu.
- ▶ Fiecare unitate de procesare va acorda prioritate celeilalte, după ce iese din secțiunea critică.
- ▶ O variabilă partajată *prioritate* conține identificatorul (indicele) unității de procesare care este prioritară momentan; *prioritate* este inițializată cu 0, adică, inițial p_0 este prioritară.
- ▶ Variabila *prioritate* este citită și modificată de ambele unități de procesare.
- ▶ Unitatea de procesare prioritară joacă rolul lui p_0 din algoritmul precedent; prin urmare, aceasta va intra în secțiunea critică.
- ▶ La ieșire, va da prioritate celeilalte unități de procesare, și va juca rolul lui p_1 din algoritmul precedent. Aceasta asigură proprietatea *no lockout*.
- ▶ Algoritmul modificat îndeplinește condițiile de excludere mutuală în aceeași manieră ca și primul algoritm.
 - ▶ O unitate de procesare ridică un fanion și apoi inspectează fanionul celeilalte unități de procesare;
 - ▶ Cel puțin una dintre unitățile de procesare observă că fanionul celeilalte unități de procesare este ridicat și evită să intre în secțiunea critică.

Excludere mutuală cu regiștri cu valori mărginite, pentru două unități de procesare, fără flămânzire – pseudocod p_0

► *Premise:* Inițial, $vreau[0] = vreau[1] = 0$, $prioritate = 0$

Algoritm 44: Pseudocod pentru unitatea de procesare p_0

⟨Entry⟩

```
1: procedure ENTRY( $vreau, prioritate$ )
2:    $vreau[0] \leftarrow 0$ 
3:   asteapta pana cand  $vreau[1] = 0$  sau  $prioritate = 0$ 
4:    $vreau[0] \leftarrow 1$ 
5:   if  $prioritate = 1$  then
6:     if  $vreau[1] = 1$  then
7:       go to linia 2
8:     else
9:       asteapta pana cand  $vreau[1] = 0$ 
```

⟨Critical⟩

⟨Exit⟩

```
1: procedure EXIT( $vreau, prioritate$ )
2:    $vreau[0] \leftarrow 0$ 
3:    $prioritate \leftarrow 1$ 
```

⟨Remainder⟩

Excludere mutuală cu regiștri cu valori mărginite, pentru două unități de procesare, fără flămânzire – pseudocod p_1

► *Premise:* Inițial, $vreau[0] = vreau[1] = 0$, $prioritate = 0$

Algoritm 45: Pseudocod pentru unitatea de procesare p_1

⟨Entry⟩

```
1: procedure ENTRY( $vreau, prioritate$ )
2:    $vreau[1] \leftarrow 0$ 
3:   asteapta pana cand  $vreau[0] = 0$  sau  $prioritate = 1$ 
4:    $vreau[1] \leftarrow 1$ 
5:   if  $prioritate = 0$  then
6:     if  $vreau[0] = 1$  then
7:       go to linia 2
8:     else
9:       asteapta pana cand  $vreau[0] = 0$ 
```

⟨Critical⟩

⟨Exit⟩

```
1: procedure EXIT( $vreau, prioritate$ )
2:    $vreau[1] \leftarrow 0$ 
3:    $prioritate \leftarrow 0$ 
```

⟨Remainder⟩

Bibliografie generală

- ▶ V. Kumar, A. Grama, A. Gupta & G Karypis, Introduction to Parallel Computing: Design and Analysis of Algorithms, Addison Wesley, 2003
- ▶ H. Attiya, J. Welch, Distributed Computing: Fundamentals, Simulations and Advanced Topics, John Wiley & Sons, Inc., 2004
- ▶ K. Berman, J. Paul, Algorithms: Sequential, Parallel, and Distributed, Thomson Learning, Inc., 2005
- ▶ T. Mattson, B. Sanders, B. Massingill, Patterns for Parallel Programming, Addison-Weslwy, 2005